

数据结构与算法分析

Violet

2026-04-17

Chapter 0

目 录

I	数据结构基础	
1.	绪论	7
2.	线性表	8
3.	栈与队列	9
4.	栈与队列	10
II	树形结构与图结构	
5.	树与二叉树基础	12
6.	树的应用	13
6.1.	哈夫曼树与哈夫曼编码	13
6.2.	并查集 (Union-Find)	14
6.3.	堆的结构与操作	16
6.4.	堆的变种与应用	18
7.	图的基本概念与存储	20
8.	图的遍历与应用	21
III	查找与排序	
9.	查找技术	23
9.1.	基本概念	23
9.2.	静态查找	24
10.	树形查找结构	30
10.1.	二叉搜索树 (BST)	30
10.2.	平衡二叉树 (AVL 树)	32
10.3.	红黑树	33
10.4.	B 树与 B+树	33
10.5.	Trie 树 (前缀树)	34
10.6.	其他树形结构	35
11.	哈希表	37
11.1.	散列表基本概念	37

11.2. Hash 函数构造方法 ····· 37

11.3. 冲突处理：开放定址法 ····· 38

11.4. 冲突处理：链地址法与其他方法 ····· 39

11.5. 性能分析与扩容策略 ····· 39

11.6. 一致性哈希算法 ····· 40

11.7. 布隆过滤器 ····· 41

11.8. 工程应用 ····· 41

11.9. 查找算法比较与选择 ····· 42

12. 内部排序算法 ····· 44

13. 高级排序与外部排序 ····· 45

IV 算法设计与分析

14. 递归、枚举与模拟 ····· 47

14.1. 递归的基本概念与设计原则 ····· 47

14.2. 递归算法的执行过程与调用栈 ····· 48

14.3. 递归树与递归复杂度分析 ····· 48

14.4. 尾递归优化与递归转迭代 ····· 49

14.5. 记忆化搜索 (Top-down DP) ····· 50

14.6. 枚举算法的设计与优化技巧 ····· 51

14.7. 模拟算法的应用场景与实现要点 ····· 52

15. 分治策略 ····· 55

16. 贪心算法 ····· 56

16.1. 贪心算法的基本思想 ····· 56

16.2. 贪心算法的正确性证明 ····· 57

16.3. 区间类问题 ····· 58

16.4. 背包类问题 ····· 59

16.5. 哈夫曼编码 ····· 60

16.6. 其他经典贪心问题 ····· 61

17. 动态规划 ····· 63

17.1. 基本概念 ····· 63

17.2. 线性 DP ····· 63

17.3. 背包问题 ····· 67

17.4. 区间 DP ····· 69

17.5. 树形 DP ····· 71

17.6. 状态压缩 DP ····· 73

17.7. 数位 DP ····· 75

17.8. DP 优化技巧总结 ····· 77

17.9. 线性 DP 补充 ····· 78

17.10 概率 DP 与期望 DP 80

17.11 DP 建模思路总结 81

17.12 DP 问题分类速查 83

18. 回溯法与分支限界 85

18.1. 回溯法的基本思想 85

18.2. 回溯法的框架与剪枝技巧 86

18.3. 经典回溯问题 87

18.4. 子集与排列问题 88

18.5. 分支限界法的基本概念 90

18.6. 分支限界的应用 91

18.7. 回溯法与分支限界的比较 93

19. 搜索优化与高级技巧 94

20. 其他算法技巧 95

V	高级数据结构
21. 线段树与树状数组	97
22. 平衡树进阶	98
23. 分块与莫队算法	99
24. 并查集与高级应用	100
25. 字符串高级数据结构	101
26. 图论高级算法	102
27. 网络流与匹配	103
28. 其他高级数据结构	104



数据结构基础

1. 绪论 ·····	7
2. 线性表 ·····	8
3. 栈与队列 ·····	9
4. 栈与队列 ·····	10

Chapter 1

绪论

Chapter 2

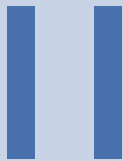
线性表

Chapter 3

栈与队列

Chapter 4

栈与队列



树形结构与图结构

5.	树与二叉树基础	12
6.	树的应用	13
7.	图的基本概念与存储	20
8.	图的遍历与应用	21

Chapter 5

树与二叉树基础

Chapter 6

树的应用

NOTE

本章介绍树的经典应用，包括哈夫曼编码、并查集、堆等数据结构。

1 哈夫曼树与哈夫曼编码

哈夫曼树（Huffman Tree）是一种带权路径长度最短的二叉树，也称为最优二叉树。

1.1 基本概念

带权路径长度（WPL）：

$$WPL = \sum_{i=1}^n w_i \cdot l_i$$

其中 w_i 是第 i 个叶子节点的权值， l_i 是从根到该叶子节点的路径长度。

1.2 哈夫曼树的构造算法

1. 将 n 个权值看作 n 棵只有根节点的二叉树，构成森林 F
2. 从 F 中选取两棵根节点权值最小的树作为左右子树，构造一棵新的二叉树
3. 新二叉树的根节点权值为左右子树根节点权值之和
4. 从 F 中删除这两棵树，将新树加入 F
5. 重复步骤 2-4，直到 F 中只剩一棵树

1.3 哈夫曼编码

哈夫曼编码是一种前缀编码，用于数据压缩。

编码规则：

- 从根到叶子节点的路径上，左分支标记为 0，右分支标记为 1
- 每个叶子节点对应的二进制串即为该字符的哈夫曼编码

特点：

- 前缀性质：任一字符的编码都不是另一字符编码的前缀
- 最优性：平均编码长度最短

1.4 应用示例

```
1 // 哈夫曼树节点
2 struct HuffmanNode {
3     char ch;
```

 C++

```

4     int freq;
5     HuffmanNode *left, *right;
6
7     HuffmanNode(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
8 };
9
10 // 比较函数（用于优先队列）
11 struct Compare {
12     bool operator()(HuffmanNode* a, HuffmanNode* b) {
13         return a->freq > b->freq;
14     }
15 };
16
17 // 构建哈夫曼树
18 HuffmanNode* buildHuffmanTree(map<char, int>& freqMap) {
19     priority_queue<HuffmanNode*, vector<HuffmanNode*>, Compare> pq;
20
21     for (auto& [ch, freq] : freqMap) {
22         pq.push(new HuffmanNode(ch, freq));
23     }
24
25     while (pq.size() > 1) {
26         HuffmanNode* left = pq.top(); pq.pop();
27         HuffmanNode* right = pq.top(); pq.pop();
28
29         HuffmanNode* newNode = new HuffmanNode('\0', left->freq + right->freq);
30         newNode->left = left;
31         newNode->right = right;
32         pq.push(newNode);
33     }
34
35     return pq.top();
36 }

```

TIP

哈夫曼编码广泛应用于文件压缩（如 ZIP、GZIP）、图像压缩（如 JPEG）等领域。

2 并查集（Union-Find）

并查集是一种树型数据结构，用于处理不相交集合并及查询问题。

2.1 基本操作

1 初始化（MakeSet）

```

1 vector<int> parent, rank_;
2
3 void makeSet(int n) {
4     parent.resize(n);
5     rank_.resize(n, 0);

```

 C++

```
6   for (int i = 0; i < n; ++i) {
7       parent[i] = i;
8   }
9 }
```

查找 (Find)

```
1 int find(int x) {
2     if (parent[x] != x) {
3         parent[x] = find(parent[x]); // 路径压缩
4     }
5     return parent[x];
6 }
```

合并 (Union)

```
1 void unionSets(int x, int y) {
2     int rootX = find(x);
3     int rootY = find(y);
4
5     if (rootX == rootY) return;
6
7     // 按秩合并
8     if (rank_[rootX] < rank_[rootY]) {
9         parent[rootX] = rootY;
10    } else if (rank_[rootX] > rank_[rootY]) {
11        parent[rootY] = rootX;
12    } else {
13        parent[rootY] = rootX;
14        rank_[rootX]++;
15    }
16 }
```

2.2 优化技术

路径压缩

在查找过程中，将路径上的所有节点直接连接到根节点。

效果：大幅降低树的高度，使后续查找更快

按秩合并

总是将较矮的树连接到较高的树下。

效果：保持树的平衡，避免退化为链表

2.3 时间复杂度

- 平摊时间复杂度： $O(\alpha(n))$ ，其中 α 是反阿克曼函数
- 实际表现：几乎为常数时间 $O(1)$

NOTE

并查集广泛应用于图的连通性判断、最小生成树（Kruskal 算法）、社交网络分析等场景。

3 堆的结构与操作

堆（Heap）是一种完全二叉树，满足堆序性，常用于实现优先队列。

3.1 堆的性质

完全二叉树

堆是一棵完全二叉树，可以用数组高效表示。

堆序性

- **最大堆**：父节点的值大于或等于子节点的值
- **最小堆**：父节点的值小于或等于子节点的值

3.2 数组实现

假设堆使用数组 `arr[]` 表示，节点编号从 0 开始：

- 父节点索引： $\text{parent}(i) = \frac{i-1}{2}$
- 左子节点索引： $\text{left}(i) = 2 * i + 1$
- 右子节点索引： $\text{right}(i) = 2 * i + 2$

NOTE

数组实现避免了显式的指针操作，空间效率高，缓存友好性好。

3.3 基本操作

插入操作（上浮）

```
1 void insert(vector<int>& heap, int val) {  
2     heap.push_back(val);  
3     int i = heap.size() - 1;  
4  
5     // 上浮调整  
6     while (i > 0) {  
7         int parent = (i - 1) / 2;  
8         if (heap[i] < heap[parent]) { // 最小堆  
9             swap(heap[i], heap[parent]);  
10            i = parent;  
11        } else {  
12            break;  
13        }  
14    }  
15 }
```

时间复杂度： $O(\log n)$

删除堆顶（下沉）

```
1  int extractMin(vector<int>& heap) {  
2      if (heap.empty()) throw runtime_error("Heap is empty");  
3  
4      int minVal = heap[0];  
5      heap[0] = heap.back();  
6      heap.pop_back();  
7  
8      // 下沉调整  
9      int i = 0;  
10     int n = heap.size();  
11  
12     while (true) {  
13         int left = 2 * i + 1;  
14         int right = 2 * i + 2;  
15         int smallest = i;  
16  
17         if (left < n && heap[left] < heap[smallest]) {  
18             smallest = left;  
19         }  
20         if (right < n && heap[right] < heap[smallest]) {  
21             smallest = right;  
22         }  
23  
24         if (smallest != i) {  
25             swap(heap[i], heap[smallest]);  
26             i = smallest;  
27         } else {  
28             break;  
29         }  
30     }  
31  
32     return minVal;  
33 }
```

时间复杂度： $O(\log n)$

建堆操作

```
1  void heapify(vector<int>& arr, int n, int i) {  
2      int smallest = i;  
3      int left = 2 * i + 1;  
4      int right = 2 * i + 2;  
5  
6      if (left < n && arr[left] < arr[smallest]) {  
7          smallest = left;  
8      }  
9      if (right < n && arr[right] < arr[smallest]) {  
10         smallest = right;  
11     }  
12  
13     if (smallest != i) {
```

```
14     swap(arr[i], arr[smallest]);
15     heapify(arr, n, smallest);
16 }
17 }
18
19 void buildHeap(vector<int>& arr) {
20     int n = arr.size();
21
22     // 从最后一个非叶子节点开始下沉
23     for (int i = n / 2 - 1; i >= 0; --i) {
24         heapify(arr, n, i);
25     }
26 }
```

时间复杂度: $O(n)$ (比逐个插入的 $O(n \log n)$ 更优)

TIP

建堆操作的时间复杂度是 $O(n)$ 而非 $O(n \log n)$, 这是因为大部分节点位于树的底层, 下沉操作的代价很小。

4 堆的变种与应用

4.1 二项堆 (Binomial Heap)

二项堆是由多棵二项树组成的森林, 支持高效的合并操作。

特点:

- 合并操作: $O(\log n)$
- 插入、删除最小值: $O(\log n)$
- 减小键值: $O(\log n)$

4.2 斐波那契堆 (Fibonacci Heap)

斐波那契堆是一种更复杂的堆结构, 具有更好的平摊性能。

特点:

- 插入、合并、减小键值: $O(1)$ 平摊时间
- 删除最小值: $O(\log n)$ 平摊时间

应用:

- Dijkstra 最短路径算法
- Prim 最小生成树算法

CAUTION

斐波那契堆虽然理论性能好, 但常数因子大, 实际应用中二项堆或普通二叉堆可能更高效。

4.3 堆的应用场景

| 优先队列

堆是优先队列的标准实现方式。

应用：

- 任务调度
- 事件驱动模拟
- 图算法（Dijkstra、Prim）

| 堆排序

基于堆的排序算法，时间复杂度稳定为 $O(n \log n)$ 。

| Top K 问题

使用大小为 K 的最小堆可以高效找到最大的 K 个元素。

时间复杂度： $O(n \log k)$



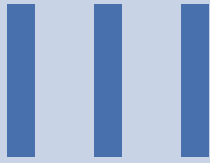
本章介绍了树的经典应用：哈夫曼编码用于数据压缩，并查集用于集合合并与查询，堆用于优先队列和排序。这些数据结构在实际工程中有着广泛的应用。

Chapter 7

图的基本概念与存储

Chapter 8

图的遍历与应用



查找与排序

9. 查找技术	23
10. 树形查找结构	30
11. 哈希表	37
12. 内部排序算法	44
13. 高级排序与外部排序	45

Chapter 9

查找技术

查找（Searching）是在数据集合中寻找满足特定条件的数据元素的过程。查找算法的效率直接影响程序的性能。

NOTE

查找算法的核心评价指标是**平均查找长度**（ASL, Average Search Length），即查找成功或失败时需要进行的關鍵字比较次数的期望值。

1 基本概念

1.1 查找表

查找表是由同一类型的数据元素（或记录）构成的集合。

静态查找表：只做查找操作，不改变表的内容

动态查找表：在查找过程中同时进行插入或删除操作

多关键字查找：数据可能存在多个关键字，查找操作需要指定查询条件

1.2 关键字

关键字是数据元素中某个数据项的值，用以标识一个数据元素。

主关键字：可以唯一标识一个记录的关键字

次关键字：不能唯一标识记录的关键字

1.3 平均查找长度（ASL）

ASL 是衡量查找算法效率的主要指标：

$$ASL = \sum_{i=1}^n p_i \cdot c_i$$

其中：

- n ：表中记录个数
- p_i ：查找第 i 个记录的概率
- c_i ：找到第 i 个记录所需的比较次数

TIP

通常假设查找每个记录的概率相等，即 $p_i = \frac{1}{n}$ 。

2 静态查找

静态查找表在查找过程中不发生变化，适合使用以下算法。

2.1 顺序查找

顺序查找是最基本的查找方法，从表的一端开始，逐个将记录的关键字与给定值进行比较。

基本实现

```
1 int sequentialSearch(vector<int>& arr, int target) {  
2     for (int i = 0; i < arr.size(); ++i) {  
3         if (arr[i] == target) {  
4             return i; // 找到，返回索引  
5         }  
6     }  
7     return -1; // 未找到  
8 }
```

时间复杂度：

- 最好情况： $O(1)$ （第一个元素就是目标）
- 最坏情况： $O(n)$ （最后一个元素或不存在）
- 平均情况： $O(n)$

ASL：

- 查找成功： $ASL = \frac{n+1}{2}$
- 查找失败： $ASL = n$

哨兵优化

通过在数组开头设置“哨兵”，可以减少每次循环中的边界检查。

```
1 int sequentialSearchWithSentinel(vector<int>& arr, int target) {  
2     int n = arr.size();  
3     arr.insert(arr.begin(), target); // 在开头插入哨兵  
4  
5     int i = n;  
6     while (arr[i] != target) {  
7         --i;  
8     }  
9  
10    arr.erase(arr.begin()); // 恢复原数组  
11    return i == 0 ? -1 : i - 1; // i=0表示未找到  
12 }
```

NOTE

哨兵优化虽然减少了比较次数，但需要修改原数组或使用额外空间，实际应用中需权衡利弊。

2.2 折半查找（二分查找）

折半查找要求查找表必须是有序的，通过不断将查找区间缩小一半来提高效率。

基本实现

```
1  int binarySearch(vector<int>& arr, int target) {
2      int left = 0, right = arr.size() - 1;
3
4      while (left <= right) {
5          int mid = left + (right - left) / 2; // 防止溢出
6
7          if (arr[mid] == target) {
8              return mid;
9          } else if (arr[mid] < target) {
10             left = mid + 1;
11          } else {
12             right = mid - 1;
13          }
14      }
15
16      return -1; // 未找到
17  }
```

时间复杂度： $O(\log n)$

ASL： $ASL \approx \log_2(n + 1) - 1$

CAUTION

计算 mid 时使用 `left + (right - left) / 2` 而非 `(left + right) / 2`，可以避免整数溢出。

二分查找的变体

查找第一个等于目标值的元素

```
1  int findFirstEqual(vector<int>& arr, int target) {
2      int left = 0, right = arr.size() - 1;
3      int result = -1;
4
5      while (left <= right) {
6          int mid = left + (right - left) / 2;
7
8          if (arr[mid] == target) {
9              result = mid;
10             right = mid - 1; // 继续向左查找
11          } else if (arr[mid] < target) {
12             left = mid + 1;
13          } else {
14             right = mid - 1;
15          }
16      }
17
18      return result;
19  }
```

查找最后一个等于目标值的元素

```
1  int findLastEqual(vector<int>& arr, int target) {  
2      int left = 0, right = arr.size() - 1;  
3      int result = -1;  
4  
5      while (left <= right) {  
6          int mid = left + (right - left) / 2;  
7  
8          if (arr[mid] == target) {  
9              result = mid;  
10             left = mid + 1; // 继续向右查找  
11         } else if (arr[mid] < target) {  
12             left = mid + 1;  
13         } else {  
14             right = mid - 1;  
15         }  
16     }  
17  
18     return result;  
19 }
```

查找第一个大于等于目标值的元素（下界）

```
1  int lowerBound(vector<int>& arr, int target) {  
2      int left = 0, right = arr.size();  
3  
4      while (left < right) {  
5          int mid = left + (right - left) / 2;  
6  
7          if (arr[mid] < target) {  
8              left = mid + 1;  
9          } else {  
10             right = mid;  
11         }  
12     }  
13  
14     return left; // 返回插入位置  
15 }
```

查找第一个大于目标值的元素（上界）

```
1  int upperBound(vector<int>& arr, int target) {  
2      int left = 0, right = arr.size();  
3  
4      while (left < right) {  
5          int mid = left + (right - left) / 2;  
6  
7          if (arr[mid] <= target) {  
8              left = mid + 1;  
9          } else {  
10             right = mid;  
11         }  
12     }  
13  
14     return left;  
15 }
```

```
11     }
12 }
13
14 return left; // 返回插入位置
15 }
```

TIP

C++ STL 提供了 `lower_bound` 和 `upper_bound` 函数，可以直接使用。

二分查找的应用

二分答案

当问题的答案具有单调性时，可以通过二分查找来确定答案。

典型问题：

- 最小化最大值
- 最大化最小值
- 求满足条件的最小/最大值

解题框架：

```
1 bool check(int mid) {
2     // 判断mid是否满足条件
3 }
4
5 int binarySearchAnswer(int left, int right) {
6     while (left < right) {
7         int mid = left + (right - left) / 2;
8
9         if (check(mid)) {
10             right = mid; // 答案在左半部分
11         } else {
12             left = mid + 1; // 答案在右半部分
13         }
14     }
15
16     return left;
17 }
```



2.3 分块查找（索引顺序查找）

分块查找结合了顺序查找和折半查找的优点，将表分成若干块，块间有序，块内无序。

算法思想

1. 建立索引表，记录每块的最大关键字和起始位置
2. 在索引表中用折半查找确定目标所在的块
3. 在块内用顺序查找找到目标

性能分析

设表长为 n ，分为 b 块，每块 s 个元素 ($n = b \times s$):

- 索引表查找: $O(\log b)$ 或 $O(b)$
- 块内查找: $O(s)$
- 总 ASL: $O(\log b + s)$ 或 $O(b + s)$

当 $b = s = \sqrt{n}$ 时, ASL 最小, 为 $O(\sqrt{n})$ 。

NOTE

分块查找适用于动态查找表, 插入和删除操作比折半查找更方便。

2.4 插值查找

插值查找是折半查找的改进, 根据目标值在有序表中的可能位置进行调整。

算法思想

将折半查找的中点计算公式:

$$mid = \text{frac}\{left + right\}\{2\}$$

改为:

$$mid = left + \text{frac}\{target - arr[left]\}\{arr[right] - arr[left]\} \cdot (right - left)$$

适用场景

- 表长较大
- 关键字分布比较均匀

时间复杂度:

- 平均情况: $O(\log \log n)$
- 最坏情况: $O(n)$ (分布极不均匀时)

CAUTION

插值查找在关键字分布不均匀时性能可能退化, 需谨慎使用。

2.5 斐波那契查找

斐波那契查找利用斐波那契数列来分割查找区间, 避免使用除法运算。

算法思想

使用斐波那契数列 $F[k]$ 来分割区间, 使得:

- 左子区间长度: $F[k-1] - 1$
- 右子区间长度: $F[k-2] - 1$

特点

- 只使用加减法, 不涉及乘除运算
- 在某些硬件平台上可能更快

- 实际应用较少

TIP

在实际开发中，折半查找已经足够高效且易于理解，推荐使用折半查找。

Chapter 10

树形查找结构

树形查找结构利用树的层次组织数据，支持高效的动态查找、插入和删除操作。

NOTE

本章介绍常见的树形查找结构，包括二叉搜索树、平衡二叉树（AVL）、红黑树、B 树/B+树、Trie 树等。

1 二叉搜索树（BST）

二叉搜索树是一种特殊的二叉树，满足以下性质：

- 若左子树非空，则左子树上所有节点的值均小于根节点的值
- 若右子树非空，则右子树上所有节点的值均大于根节点的值
- 左右子树也分别是二叉搜索树

1.1 BST 的基本操作

查找

```
1  TreeNode* searchBST(TreeNode* root, int val) {  
2      if (!root || root->val == val) {  
3          return root;  
4      }  
5  
6      if (val < root->val) {  
7          return searchBST(root->left, val);  
8      } else {  
9          return searchBST(root->right, val);  
10     }  
11 }
```

时间复杂度：

- 最好情况： $O(\log n)$ （平衡树）
- 最坏情况： $O(n)$ （退化为链表）

插入

```
1  TreeNode* insertBST(TreeNode* root, int val) {  
2      if (!root) {  
3          return new TreeNode(val);  
4      }  
5  
6      if (val < root->val) {  
7          root->left = insertBST(root->left, val);  
8      } else {  
9          root->right = insertBST(root->right, val);  
10     }  
11 }
```

```
8     } else if (val > root->val) {
9         root->right = insertBST(root->right, val);
10    }
11
12    return root;
13 }
```

删除

删除操作分三种情况：

1. 叶子节点：直接删除
2. 只有一个子节点：用子节点替代
3. 有两个子节点：用中序后继（右子树最小节点）替代

```
1  TreeNode* deleteBST(TreeNode* root, int val) {
2      if (!root) return nullptr;
3
4      if (val < root->val) {
5          root->left = deleteBST(root->left, val);
6      } else if (val > root->val) {
7          root->right = deleteBST(root->right, val);
8      } else {
9          // 找到要删除的节点
10         if (!root->left) {
11             TreeNode* temp = root->right;
12             delete root;
13             return temp;
14         } else if (!root->right) {
15             TreeNode* temp = root->left;
16             delete root;
17             return temp;
18         } else {
19             // 有两个子节点，找中序后继
20             TreeNode* successor = findMin(root->right);
21             root->val = successor->val;
22             root->right = deleteBST(root->right, successor->val);
23         }
24     }
25
26     return root;
27 }
28
29 TreeNode* findMin(TreeNode* node) {
30     while (node->left) {
31         node = node->left;
32     }
33     return node;
34 }
```

1.2 BST 的性能分析

BST 的性能取决于树的形态：

- 最好情况：完全平衡，高度为 $O(\log n)$
- 最坏情况：退化为链表，高度为 $O(n)$

CAUTION

BST 在插入顺序不利时会严重退化，实际应用中应使用平衡二叉搜索树。

1.3 BST 的应用场景

- 教学演示：理解二叉搜索的基本原理
- 小规模数据：数据量小且随机插入时表现良好
- 其他平衡树的基础：AVL、红黑树等都是 BST 的改进版本

2 平衡二叉树 (AVL 树)

AVL 树是一种自平衡的二叉搜索树，任意节点的左右子树高度差不超过 1。

2.1 AVL 树的定义与平衡因子

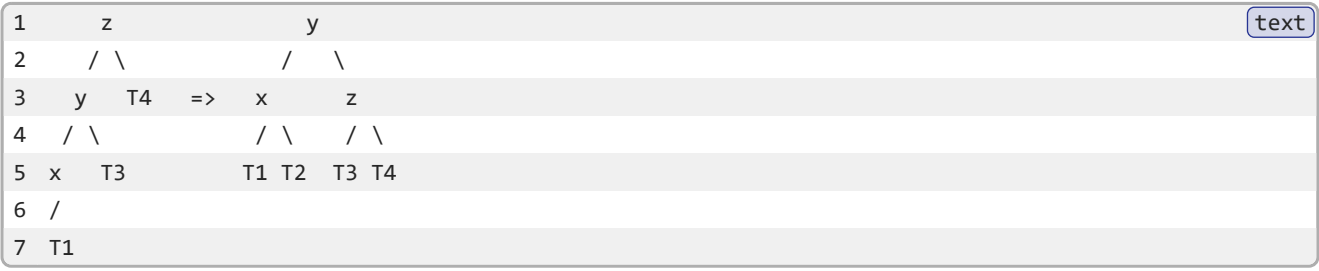
平衡因子 = 左子树高度 - 右子树高度

AVL 树要求所有节点的平衡因子为 -1、0 或 1。

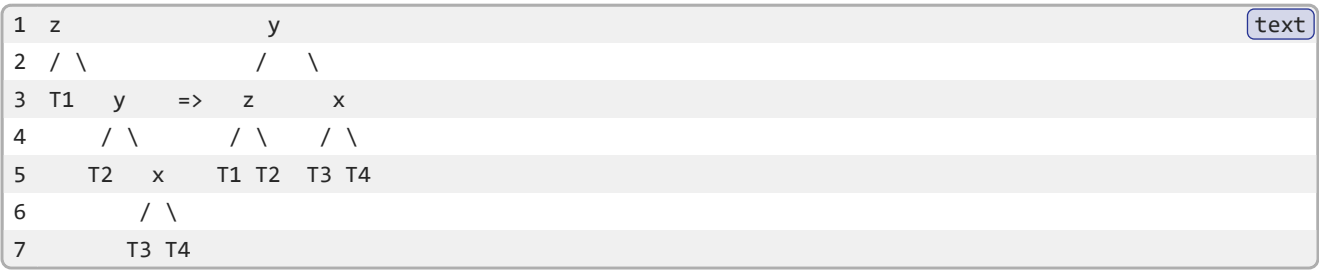
2.2 AVL 树的旋转操作

当插入或删除节点导致树不平衡时，需要通过旋转来恢复平衡。

LL 旋转 (右旋)



RR 旋转 (左旋)



LR 旋转 (先左后右)

先对左子节点进行 RR 旋转，再对根节点进行 LL 旋转。

I RL 旋转（先右后左）

先对右子节点进行 LL 旋转，再对根节点进行 RR 旋转。

2.3 AVL 树的性能分析

- 查找、插入、删除的时间复杂度均为 $O(\log n)$
- 保持严格平衡，查询效率高
- 插入和删除可能需要多次旋转

NOTE

AVL 树适合读多写少的场景，如数据库索引。

3 红黑树

红黑树是一种近似平衡的二叉搜索树，通过颜色约束来保证平衡。

3.1 红黑树的五条性质

1. 每个节点要么是红色，要么是黑色
2. 根节点是黑色
3. 每个叶子节点（NIL）是黑色
4. 如果一个节点是红色，则它的两个子节点都是黑色
5. 从任一节点到其每个叶子的所有简单路径都包含相同数目的黑色节点

3.2 红黑树的性能特点

- 查找、插入、删除的时间复杂度均为 $O(\log n)$
- 插入和删除最多只需要常数次旋转
- 比 AVL 树更宽松，适合频繁插入删除的场景

TIP

C++ STL 中的 `map` 和 `set`、Java 中的 `TreeMap` 和 `TreeSet` 都基于红黑树实现。

3.3 AVL vs 红黑树对比

特性	AVL 树	红黑树
平衡性	严格平衡	近似平衡
查询效率	更高	稍低
插入/删除	可能需要多次旋转	最多常数次旋转
适用场景	读多写少	读写均衡
实现难度	较复杂	相对简单

未完待续...

4 B 树与 B+树

B 树和 B+树是多路平衡搜索树，专为磁盘存储设计。

4.1 B 树的定义与性质

m 阶 B 树的性质：

- 每个节点最多有 m 个子节点
- 除根节点外，每个节点至少有 $\lceil \frac{m}{2} \rceil$ 个子节点
- 所有叶子节点在同一层
- 节点中的关键字按升序排列

4.2 B 树的操作

B 树支持查找、插入、删除操作，其中插入和删除可能涉及节点的分裂与合并。

应用场景：文件系统、数据库索引

4.3 B+树的结构特点

B+树是 B 树的变种，具有以下特点：

- 非叶子节点只存储索引，不存储数据
- 所有数据都存储在叶子节点
- 叶子节点之间用链表连接，便于范围查询

4.4 B+树的优势

- 更适合范围查询
- 磁盘 I/O 次数更少
- 遍历效率更高

NOTE

MySQL 的 InnoDB 存储引擎使用 B+树作为索引结构。

5 Trie 树（前缀树）

Trie 树是一种用于字符串检索的多叉树结构。

5.1 Trie 树的结构特点

- 根节点不包含字符
- 每条边代表一个字符
- 从根到某节点的路径上的字符连接起来，即为该节点对应的字符串
- 每个节点的所有子节点包含的字符都不相同

5.2 Trie 树的基本操作

I 插入

```
1 struct TrieNode {
```



```
2     TrieNode* children[26];
3     bool isEnd;
4
5     TrieNode() {
6         for (int i = 0; i < 26; ++i) {
7             children[i] = nullptr;
8         }
9         isEnd = false;
10    }
11 };
12
13 void insert(TrieNode* root, string word) {
14     TrieNode* node = root;
15     for (char c : word) {
16         int idx = c - 'a';
17         if (!node->children[idx]) {
18             node->children[idx] = new TrieNode();
19         }
20         node = node->children[idx];
21     }
22     node->isEnd = true;
23 }
```

查找

```
1 bool search(TrieNode* root, string word) {
2     TrieNode* node = root;
3     for (char c : word) {
4         int idx = c - 'a';
5         if (!node->children[idx]) {
6             return false;
7         }
8         node = node->children[idx];
9     }
10    return node->isEnd;
11 }
```



5.3 Trie 树的应用场景

- 自动补全
- 拼写检查
- IP 路由表查找
- 词频统计

时间复杂度: $O(m)$, 其中 m 是字符串长度

TIP

Trie 树的空间消耗较大, 可以使用压缩 Trie (基数树) 或双数组 Trie 来优化。

6 其他树形结构

6.1 后缀树与后缀数组

后缀树是一种压缩 Trie 树，存储字符串的所有后缀。

应用：

- 字符串匹配
- 最长重复子串
- 最长公共子串

后缀数组：后缀树的数组实现，空间效率更高。

6.2 线段树

线段树是一种二叉树，用于高效处理区间查询和更新。

应用：

- 区间求和
- 区间最值
- 区间更新

时间复杂度：查询和更新均为 $O(\log n)$

6.3 树状数组（Fenwick Tree）

树状数组是一种简洁的数据结构，用于维护前缀和。

优势：

- 代码简洁
- 常数小
- 空间效率高

应用：

- 前缀和查询
- 单点更新
- 逆序对计数



本章介绍了常见的树形查找结构。BST 是基础，AVL 和红黑树通过平衡机制保证性能，B 树/B+树专为磁盘优化，Trie 树用于字符串处理。不同的树适用于不同的场景，选择合适的树结构可以显著提升系统性能。

Chapter 11

哈希表

哈希表通过哈希函数将关键字映射到表中的位置，实现快速查找。

NOTE

本章介绍哈希表的基本原理、Hash 函数构造、冲突处理、性能分析以及工程应用。

1 散列表基本概念

1.1 什么是散列表

散列表（Hash Table）是一种基于键值对（Key-Value）的数据结构，通过哈希函数将键映射到表中的位置，从而实现快速的插入、删除和查找操作。

1.2 核心思想

1. 哈希函数：将任意长度的键映射为固定范围的整数（哈希地址）
2. 冲突处理：当多个键映射到同一位置时，采用特定策略解决
3. 装填因子：衡量哈希表的拥挤程度，影响性能

1.3 哈希表的特点

优点：

- 平均情况下，插入、删除、查找的时间复杂度为 $O(1)$
- 实现简单，应用广泛

缺点：

- 最坏情况下退化为 $O(n)$
- 不支持范围查询和排序
- 需要良好的哈希函数和冲突处理策略

未完待续...

2 Hash 函数构造方法

Hash 函数的目标是将键均匀地分布到哈希表中，减少冲突。

2.1 直接定址法

$h(\text{key}) = \text{key}$ 或 $h(\text{key}) = a * \text{key} + b$

优点：简单、无冲突

缺点：只适用于关键字分布连续的情况

2.2 除留余数法

$h(\text{key}) = \text{key} \bmod p$, 其中 $p \leq m$ (表长)

优点：最简单、最常用

注意：p 最好选择素数

TIP

除留余数法是最常用的哈希函数构造方法，实际应用中优先选择。

2.3 其他构造方法

数字分析法

抽取关键字中分布均匀的若干位作为哈希地址。

适用场景：关键字位数较多，且某些位分布均匀

平方取中法

取关键字平方后的中间几位作为哈希地址。

优点：适用于关键字分布不均匀的情况

折叠法

将关键字分割成位数相同的几部分，然后取叠加和作为哈希地址。

适用场景：关键字位数很多

随机数法

$h(\text{key}) = \text{random}(\text{key})$

适用场景：关键字长度不等

未完待续...

3 冲突处理：开放定址法

当发生冲突时，按照某种探测序列寻找下一个空闲位置。

3.1 线性探测法

$h_i(\text{key}) = (h(\text{key}) + i) \bmod m, i = 1, 2, \dots, m - 1$

优点：实现简单

缺点：容易产生聚集现象

3.2 二次探测法

$$h_{i(\text{key})} = (h(\text{key}) + i^2) \bmod m \text{ 或 } h_{i(\text{key})} = (h(\text{key}) - i^2) \bmod m$$

优点：减少了一次聚集

缺点：仍然可能存在二次聚集

3.3 伪随机探测法

$$h_{i(\text{key})} = (h(\text{key}) + \text{pseudo_random}(i)) \bmod m$$

优点：进一步减少聚集

未完待续...

4 冲突处理：链地址法与其他方法

4.1 链地址法（拉链法）

将所有哈希地址相同的记录链接成一个单链表。

优点：

- 不会产生聚集
- 删除操作简单
- 装填因子可以大于 1

缺点：

- 需要额外的指针空间
- 缓存友好性较差

NOTE

Java 的 `HashMap`、C++ 的 `unordered_map` 都使用链地址法处理冲突。

4.2 再哈希法

当发生冲突时，使用另一个哈希函数计算新的地址。

优点：不易产生聚集

缺点：增加计算时间

4.3 建立公共溢出区

将哈希表分为基本表和溢出表两部分，所有冲突的记录都放入溢出表。

未完待续...

5 性能分析与扩容策略

5.1 装填因子与性能分析

哈希表的性能取决于装填因子 α ：

$$\alpha = \frac{n}{m} = \frac{\text{表中记录数}}{\text{哈希表长度}}$$

查找成功的平均查找长度：

- 链地址法：ASL $\approx 1 + \frac{\alpha}{2}$
- 线性探测法：ASL $\approx \frac{1}{2}(1 + \frac{1}{1 - \alpha})$

查找失败的平均查找长度：

- 链地址法：ASL $\approx \alpha + e^{-\alpha}$
- 线性探测法：ASL $\approx \frac{1}{2}(1 + \frac{1}{(1 - \alpha)^2})$

CAUTION

装填因子越大，冲突概率越高，查找效率越低。一般建议 $\alpha \leq 0.75$ 。

5.2 哈希表的扩容与 Rehash

当装填因子超过阈值时，需要扩大哈希表容量并重新哈希所有元素。

扩容策略：

- 通常将表长扩大为原来的 2 倍
- 选择新的表长为素数
- 重新计算所有元素的哈希值并插入新表

时间复杂度： $O(n)$ ，但平摊到每次插入操作为 $O(1)$

未完待续...

6 一致性哈希算法

一致性哈希用于分布式缓存系统，解决节点增减时的数据迁移问题。

6.1 核心思想

- 将哈希空间组织成一个环
- 节点和数据都映射到环上
- 数据存储在顺时针方向的第一个节点上

6.2 优点

- 节点增减时，只有少量数据需要迁移
- 负载均衡效果好

NOTE

Redis 集群、Memcached 等分布式缓存系统都使用一致性哈希。

7 布隆过滤器

布隆过滤器是一种空间高效的概率型数据结构，用于判断元素是否在集合中。

7.1 工作原理

- 使用多个哈希函数将元素映射到位数组的多个位置
- 查询时，检查所有对应位置是否都为 1

7.2 特点

- 可能存在误判（假阳性），但不会漏判（假阴性）
- 空间效率极高
- 不支持删除操作（可使用计数布隆过滤器解决）

7.3 应用场景

- 网页去重
- 缓存穿透防护
- 垃圾邮件过滤

TIP

布隆过滤器的误判率可以通过调整位数组大小和哈希函数数量来控制。

8 工程应用

8.1 数据库索引设计

哈希索引 vs B+树索引：

特性	哈希索引	B+树索引
等值查询	$O(1)$	$O(\log n)$
范围查询	不支持	支持
排序	不支持	支持
最左前缀	不支持	支持
空间占用	较小	较大

NOTE

MySQL 的 Memory 存储引擎支持哈希索引，InnoDB 支持自适应哈希索引。

8.2 倒排索引与搜索引擎

倒排索引是搜索引擎的核心数据结构。

结构：

1 单词 -> [文档ID列表]

应用：

- 全文检索
- 关键词匹配
- 相关性排序

8.3 LRU 缓存设计与实现

LRU（Least Recently Used）缓存淘汰最近最少使用的数据。

实现方式：

- 哈希表 + 双向链表
- 哈希表提供 $O(1)$ 查找
- 双向链表维护访问顺序

时间复杂度：

- 查找： $O(1)$
- 插入： $O(1)$
- 删除： $O(1)$

本章介绍了哈希表的基本原理、Hash 函数构造、冲突处理策略、性能分析以及工程应用。哈希表作为一种高效的数据结构，在数据库、缓存、搜索引擎等领域有着广泛的应用。

9 查找算法比较与选择

9.1 各种查找算法对比

算法	平均时间复杂度	是否需要有序	适用场景
顺序查找	$O(n)$	否	小规模、无序数据
折半查找	$O(\log n)$	是	静态有序表
分块查找	$O(\sqrt{n})$	块间有序	动态查找表
BST	$O(\log n) \sim O(n)$	隐含有序	动态查找
AVL 树	$O(\log n)$	隐含有序	读多写少
红黑树	$O(\log n)$	隐含有序	频繁插入删除
B+树	$O(\log n)$	隐含有序	磁盘存储、数据库
哈希表	$O(1)$	否	等值查询
Trie 树	$O(m)$	否	字符串检索

9.2 选择策略

根据数据特点选择

- 数据量小：顺序查找即可
- 数据有序：折半查找
- 需要范围查询：B+树、平衡二叉树
- 只需等值查询：哈希表
- 字符串前缀匹配：Trie 树

根据操作频率选择

- 查多改少：AVL 树、B+树
- 查改频繁：红黑树、哈希表
- 动态插入删除：平衡二叉树、哈希表

根据存储介质选择

- 内存：哈希表、平衡二叉树
- 磁盘：B+树、B 树

本章完

Chapter 12

内部排序算法

Chapter 13

高级排序与外部排序

IV

算法设计与分析

14. 递归、枚举与模拟 ·····	47
15. 分治策略 ·····	55
16. 贪心算法 ·····	56
17. 动态规划 ·····	63
18. 回溯法与分支限界 ·····	85
19. 搜索优化与高级技巧 ·····	94
20. 其他算法技巧 ·····	95

Chapter 14

递归、枚举与模拟

NOTE

本章介绍三种基础算法思想：递归、枚举和模拟。这些思想是许多高级算法的基础。

1 递归的基本概念与设计原则

递归是一种自调用的算法设计方法，通过将问题分解为更小的子问题来求解。

1.1 递归的两个关键要素

基准情况（Base Case）

基准情况是递归的终止条件，防止无限递归。

示例：

```
1 int factorial(int n) {  
2     if (n <= 1) return 1; // 基准情况  
3     return n * factorial(n - 1);  
4 }
```

C++

递归进展（Recursive Progress）

每次递归调用必须向基准情况靠近，确保最终能够终止。

示例：

```
1 int fibonacci(int n) {  
2     if (n <= 1) return n; // 基准情况  
3     return fibonacci(n - 1) + fibonacci(n - 2); // 向基准情况靠近  
4 }
```

C++

1.2 递归的设计原则

1. 明确基准情况：确定何时停止递归
2. 缩小问题规模：每次递归处理更小的子问题
3. 相信递归：假设递归调用能正确解决子问题
4. 避免重复计算：考虑使用记忆化优化

TIP

设计递归算法时，先想清楚基准情况和递归进展，再编写代码。

2 递归算法的执行过程与调用栈

2.1 调用栈的工作原理

递归调用时，系统会为每次调用创建一个栈帧（Stack Frame），保存局部变量、参数和返回地址。

示例：计算 `factorial(4)` 的调用过程

```
1 factorial(4)
2 |─ factorial(3)
3 | |─ factorial(2)
4 | | |─ factorial(1) ← 基准情况，返回 1
5 | | |─ 返回 2 * 1 = 2
6 | |─ 返回 3 * 2 = 6
7 |─ 返回 4 * 6 = 24
```

2.2 递归深度与栈溢出

问题：递归过深会导致栈溢出（Stack Overflow）

```
1 // 危险：可能导致栈溢出
2 void deepRecursion(int n) {
3     if (n <= 0) return;
4     deepRecursion(n - 1);
5 }
6
7 // 调用 deepRecursion(100000) 可能栈溢出
```

解决方案：

- 1. 限制递归深度
- 2. 转换为迭代
- 3. 使用尾递归优化

CAUTION

大多数系统的默认栈大小为 1–8 MB，递归深度通常限制在几千到几万层。

3 递归树与递归复杂度分析

3.1 递归树方法

递归树是分析递归算法复杂度的直观工具。

示例：归并排序的递归树

```
1      T(n)          ← 第0层：1个节点，代价 cn
2      /  \
3  T(n/2) T(n/2)    ← 第1层：2个节点，代价 cn
4  / \  / \
```



```

5  T(n/4)...           ← 第2层: 4个节点, 代价 cn
6  ...

```

总代价: $T(n) = cn * \log n = O(n \log n)$

3.2 主定理 (Master Theorem)

对于形式为 $T(n) = a * T(\frac{n}{b}) + f(n)$ 的递归关系:

情况	条件	解
情况 1	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$
情况 2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$
情况 3	$f(n) = \Omega(n^{\log_b a + \epsilon})$	$T(n) = \Theta(f(n))$

示例:

- 归并排序: $T(n) = 2T(\frac{n}{2}) + O(n) \rightarrow O(n \log n)$
- 二分查找: $T(n) = T(\frac{n}{2}) + O(1) \rightarrow O(\log n)$

4 尾递归优化与递归转迭代

4.1 尾递归

尾递归是指递归调用是函数的最后一个操作。

非尾递归:

```

1 int factorial(int n) {
2     if (n <= 1) return 1;
3     return n * factorial(n - 1); // 乘法在递归调用之后
4 }

```

尾递归:

```

1 int factorialTail(int n, int acc = 1) {
2     if (n <= 1) return acc;
3     return factorialTail(n - 1, n * acc); // 递归调用是最后一步
4 }

```

4.2 尾递归优化

编译器可以将尾递归优化为循环, 避免栈增长。

优点:

- 空间复杂度从 $O(n)$ 降为 $O(1)$
- 避免栈溢出

NOTE

C++编译器 (如 GCC、Clang) 在优化模式下会自动进行尾递归优化。

4.3 递归转迭代

任何递归都可以转换为迭代，使用**显式栈**模拟调用栈。

示例：二叉树前序遍历

```
1 // 递归版本
2 void preorder(TreeNode* root) {
3     if (!root) return;
4     visit(root);
5     preorder(root->left);
6     preorder(root->right);
7 }
8
9 // 迭代版本
10 void preorderIterative(TreeNode* root) {
11     stack<TreeNode*> stk;
12     if (root) stk.push(root);
13
14     while (!stk.empty()) {
15         TreeNode* node = stk.top(); stk.pop();
16         visit(node);
17         if (node->right) stk.push(node->right);
18         if (node->left) stk.push(node->left);
19     }
20 }
```

5 记忆化搜索 (Top-down DP)

记忆化搜索是**递归 + 缓存**的组合，避免重复计算子问题。

5.1 基本原理

1. 使用递归自顶向下求解
2. 用哈希表或数组缓存已计算的结果
3. 再次遇到相同子问题时直接返回缓存值

5.2 经典示例：斐波那契数列

普通递归： $O(2^n)$

```
1 int fib(int n) {
2     if (n <= 1) return n;
3     return fib(n - 1) + fib(n - 2); // 大量重复计算
4 }
```

记忆化搜索： $O(n)$

```
1 unordered_map<int, int> memo;
2
3 int fibMemo(int n) {
4     if (n <= 1) return n;
```

```
5     if (memo.count(n)) return memo[n]; // 查缓存
6     return memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
7 }
```

5.3 应用场景

- 动态规划问题（Top-down 方式）
- 有重叠子问题的递归
- 状态空间稀疏的问题

TIP

记忆化搜索的优点是只计算需要的状态，而 DP 会计算所有状态。对于状态空间稀疏的问题，记忆化搜索更高效。

6 枚举算法的设计与优化技巧

枚举（穷举）是一种暴力搜索方法，遍历所有可能的解空间。

6.1 基本框架

```
1 for (每个可能的选择1) {
2     for (每个可能的选择2) {
3         ...
4         if (满足条件) {
5             更新最优解
6         }
7     }
8 }
```

6.2 优化技巧

缩小搜索空间

通过问题分析排除不可能的情况。

示例：两数之和

```
1 // 暴力枚举:  $O(n^2)$ 
2 for (int i = 0; i < n; ++i) {
3     for (int j = i + 1; j < n; ++j) {
4         if (nums[i] + nums[j] == target) {
5             return {i, j};
6         }
7     }
8 }
9
10 // 优化: 使用哈希表  $O(n)$ 
11 unordered_map<int, int> map;
12 for (int i = 0; i < n; ++i) {
13     int complement = target - nums[i];
```

```
14     if (map.count(complement)) {
15         return {map[complement], i};
16     }
17     map[nums[i]] = i;
18 }
```

剪枝

提前终止不可能的分支。

示例：N 皇后问题

```
1 void solveNQueens(int row) {
2     if (row == n) {
3         count++;
4         return;
5     }
6
7     for (int col = 0; col < n; ++col) {
8         if (isValid(row, col)) { // 剪枝：检查是否冲突
9             placeQueen(row, col);
10            solveNQueens(row + 1);
11            removeQueen(row, col);
12        }
13    }
14 }
```

位运算优化

使用位运算加速枚举过程。

示例：子集枚举

```
1 // 枚举集合 {0, 1, ..., n-1} 的所有子集
2 for (int mask = 0; mask < (1 << n); ++mask) {
3     // mask 的二进制表示对应一个子集
4     for (int i = 0; i < n; ++i) {
5         if (mask & (1 << i)) {
6             // 元素 i 在子集中
7         }
8     }
9 }
```

7 模拟算法的应用场景与实现要点

模拟算法是直接按照题目描述逐步执行，没有特定的优化技巧。

7.1 适用场景

- 规则明确的流程模拟
- 游戏逻辑实现
- 系统行为仿真

- 复杂的状态转换

7.2 实现要点

清晰的状态表示

```
1 struct GameState {  
2     int playerPos;  
3     int score;  
4     vector<bool> visited;  
5     // ...  
6 };
```

模块化设计

将复杂逻辑分解为小函数。

```
1 void simulate() {  
2     initialize();  
3     while (!isGameOver()) {  
4         processInput();  
5         updateState();  
6         render();  
7     }  
8 }
```

边界条件处理

特别注意特殊情况。

示例：日期模拟

```
1 void nextDay(int& year, int& month, int& day) {  
2     day++;  
3  
4     // 处理月末  
5     if (day > daysInMonth(year, month)) {  
6         day = 1;  
7         month++;  
8  
9         // 处理年末  
10        if (month > 12) {  
11            month = 1;  
12            year++;  
13        }  
14    }  
15 }
```

7.3 经典示例：约瑟夫环

```
1 int josephus(int n, int k) {  
2     if (n == 1) return 0;  
3     return (josephus(n - 1, k) + k) % n;  
}
```

```
4  }  
5  
6  // 迭代版本  
7  int josephusIterative(int n, int k) {  
8      int result = 0;  
9      for (int i = 2; i <= n; ++i) {  
10         result = (result + k) % i;  
11     }  
12     return result;  
13 }
```

本章介绍了三种基础算法思想：递归通过自调用分解问题，枚举通过遍历寻找解，模拟通过逐步执行还原过程。这些思想虽然简单，却是许多高级算法的基础。

Chapter 15

分治策略

Chapter 16

贪心算法

NOTE

贪心算法是一种在每一步选择中都采取当前最优（或最有利）的选择，从而希望导致结果是全局最优的算法。

1 贪心算法的基本思想

贪心算法通过局部最优选择来构造全局最优解。

1.1 核心特征

贪心选择性质

可以通过局部最优选择来构造全局最优解。

特点：

- 做出选择后，问题规模缩小
- 不需要回溯或重新考虑之前的选择
- 依赖于问题的特定结构

最优子结构

一个问题的最优解包含其子问题的最优解。

示例：

- 最短路径：从 A 到 C 的最短路径经过 B，则 A→B 和 B→C 也都是最短路径
- 最小生成树：去掉一条边后，剩余部分仍然是子图的最小生成树

1.2 贪心 vs 动态规划

特性	贪心算法	动态规划
选择方式	局部最优	全局最优
子问题	只解决一次	可能重复解决
回溯	不需要	需要（隐式）
适用性	特定问题	更广泛
效率	通常更高	可能较低

TIP

贪心算法比动态规划更高效，但适用范围更窄。只有当问题具有贪心选择性质时才能使用。

1.3 贪心的局限性

反例：0/1 背包问题

```
1 // 物品：(重量, 价值)
2 // A: (10, 60), B: (20, 100), C: (30, 120)
3 // 背包容量: 50
4
5 // 贪心策略 (按价值/重量比):
6 // A: 6.0, B: 5.0, C: 4.0
7 // 选择A + B = 60 + 100 = 160
8
9 // 最优解: B + C = 100 + 120 = 220
```



原因：贪心选择可能导致无法达到全局最优

2 贪心算法的正确性证明

证明贪心算法的正确性通常使用以下方法。

2.1 交换论证法 (Exchange Argument)

核心思想：假设存在一个最优解，通过交换操作将其转换为贪心解，且不降低解的质量。

步骤：

1. 假设存在最优解 O
2. 如果 O 与贪心解 G 不同，找到第一个不同的选择
3. 通过交换操作将 O 转换为 G
4. 证明交换后解的质量不降低
5. 因此 G 也是最优解

示例：活动选择问题

```
1 // 按结束时间排序后，贪心选择最早结束的活动
2 // 证明：如果最优解选择了其他活动，可以交换为贪心选择
3 // 且不会减少可选择的活动数量
```



2.2 数学归纳法

核心思想：证明贪心选择后，剩余子问题的最优解加上贪心选择构成原问题的最优解。

步骤：

1. 基准情况：问题规模为 1 时，贪心选择显然最优
2. 归纳假设：假设对于规模 $n - 1$ 的问题，贪心算法得到最优解
3. 归纳步骤：证明对于规模 n 的问题，贪心选择 + 子问题最优解 = 原问题最优解

NOTE

交换论证法更直观，数学归纳法更严谨。实际应用中常结合使用。

3 区间类问题

区间问题是贪心算法的经典应用场景。

3.1 活动选择问题

问题：从 n 个活动中选择最多的互不冲突的活动。

贪心策略：按结束时间排序，选择最早结束且与已选活动不冲突的活动。

```
1 struct Activity {
2     int start, end;
3 };
4
5 int maxActivities(vector<Activity>& acts) {
6     // 按结束时间排序
7     sort(acts.begin(), acts.end(),
8         [](const Activity& a, const Activity& b) {
9             return a.end < b.end;
10        });
11
12    int count = 1;
13    int lastEnd = acts[0].end;
14
15    for (int i = 1; i < acts.size(); ++i) {
16        if (acts[i].start >= lastEnd) {
17            count++;
18            lastEnd = acts[i].end;
19        }
20    }
21
22    return count;
23 }
```

时间复杂度： $O(n \log n)$ （排序）

3.2 区间调度

问题：给定多个区间，选择最少的点使得每个区间至少包含一个点。

贪心策略：按右端点排序，每次选择当前区间的右端点。

```
1 int minPoints(vector<pair<int, int>>& intervals) {
2     sort(intervals.begin(), intervals.end(),
3         [](const pair<int, int>& a, const pair<int, int>& b) {
4             return a.second < b.second;
5         });
6
7     int points = 1;
8     int lastPoint = intervals[0].second;
9
10    for (int i = 1; i < intervals.size(); ++i) {
11        if (intervals[i].first > lastPoint) {
```

```
12         points++;
13         lastPoint = intervals[i].second;
14     }
15 }
16
17 return points;
18 }
```

3.3 区间覆盖

问题：用最少的区间覆盖目标区间 $[0, T]$ 。

贪心策略：每次选择能覆盖当前位置且右端点最远的区间。

4 背包类问题

4.1 分数背包问题

问题：物品可以分割，求最大价值。

贪心策略：按单位重量价值排序，优先选择价值密度高的物品。

```
1  struct Item {
2      int weight;
3      int value;
4      double ratio() const {
5          return (double)value / weight;
6      }
7  };
8
9  double fractionalKnapsack(vector<Item>& items, int capacity) {
10     // 按价值密度排序
11     sort(items.begin(), items.end(),
12          [](const Item& a, const Item& b) {
13              return a.ratio() > b.ratio();
14          });
15
16     double totalValue = 0;
17     int remaining = capacity;
18
19     for (const auto& item : items) {
20         if (remaining <= 0) break;
21
22         if (item.weight <= remaining) {
23             // 全部装入
24             totalValue += item.value;
25             remaining -= item.weight;
26         } else {
27             // 部分装入
28             totalValue += item.ratio() * remaining;
29             remaining = 0;
```

 C++

```
30     }
31 }
32
33 return totalValue;
34 }
```

时间复杂度: $O(n \log n)$

NOTE

分数背包可以用贪心, 但 0/1 背包必须用动态规划。

5 哈夫曼编码

哈夫曼编码是贪心算法构造最优前缀码的经典应用。

5.1 基本原理

目标: 为字符集构造变长编码, 使得平均编码长度最短。

贪心策略: 每次合并频率最小的两个节点。

5.2 构造过程

```
1  struct HuffmanNode {
2      char ch;
3      int freq;
4      HuffmanNode *left, *right;
5
6      HuffmanNode(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
7  };
8
9  HuffmanNode* buildHuffmanTree(map<char, int>& freqMap) {
10     // 最小堆
11     auto cmp = [](HuffmanNode* a, HuffmanNode* b) {
12         return a->freq > b->freq;
13     };
14     priority_queue<HuffmanNode*, vector<HuffmanNode*>, decltype(cmp)> pq(cmp);
15
16     // 初始化
17     for (auto& [ch, freq] : freqMap) {
18         pq.push(new HuffmanNode(ch, freq));
19     }
20
21     // 贪心合并
22     while (pq.size() > 1) {
23         HuffmanNode* left = pq.top(); pq.pop();
24         HuffmanNode* right = pq.top(); pq.pop();
25
26         HuffmanNode* newNode = new HuffmanNode('\0', left->freq + right->freq);
27         newNode->left = left;
```

```

28     newNode->right = right;
29     pq.push(newNode);
30 }
31
32 return pq.top();
33 }

```

时间复杂度: $O(n \log n)$

5.3 编码示例

```

1  字符: a(45), b(13), c(12), d(16), e(9), f(5)
2
3  哈夫曼树:
4      100
5      /  \
6     a(45) 55
7         /  \
8        c(12) 43
9           /  \
10          b(13) 30
11             /  \
12            e(9) f(5)
13
14 d(16) 未显示, 实际会更复杂
15
16 编码:
17 a: 0
18 b: 101
19 c: 100
20 d: 111
21 e: 1101
22 f: 1100

```

TIP

哈夫曼编码广泛应用于文件压缩（ZIP、GZIP）、图像压缩（JPEG）等。

6 其他经典贪心问题

6.1 硬币找零

问题: 用最少的硬币凑出指定金额。

贪心策略: 优先使用面额最大的硬币。

```

1  int minCoins(vector<int>& coins, int amount) {
2      sort(coins.rbegin(), coins.rend()); // 降序
3
4      int count = 0;
5      for (int coin : coins) {

```



```
6     count += amount / coin;
7     amount %= coin;
8 }
9
10 return amount == 0 ? count : -1;
11 }
```

注意：仅当硬币面额满足特定条件时贪心才正确（如美分系统）。

6.2 任务调度

问题：最大化完成任务的数量或总价值。

贪心策略：

- 最大化数量：按截止时间排序
- 最大化价值：按价值密度排序



本章介绍了贪心算法的基本思想、正确性证明方法以及经典应用。贪心算法虽然简单高效，但需要仔细验证其正确性，避免陷入局部最优的陷阱。

Chapter 17

动态规划

动态规划 (Dynamic Programming, 简称 DP) 是一种用于解决复杂优化问题的算法思想, 它通过将问题分解为相互重叠的子问题, 并存储子问题的解 (避免重复计算) 来高效解决问题。

NOTE

动态规划基于两个重要概念: **最优子结构**和**重叠子问题**。

1 基本概念

1.1 核心思想

动态规划的核心在于以下两个关键性质:

- **最优子结构**: 一个问题的最优解包含其子问题的最优解。这意味着我们可以通过组合子问题的最优解来构造原问题的最优解。
- **重叠子问题**: 在递归求解过程中, 相同的子问题会被多次计算。动态规划通过存储这些子问题的解 (记忆化), 避免了重复计算。

1.2 基本步骤

解决动态规划问题通常遵循以下步骤:

1. **定义状态**: 找出问题中的变量, 用状态表示子问题的解
2. **确定状态转移方程**: 建立不同状态之间的关系 (递推公式)
3. **设置初始条件**: 确定最简单子问题的解 (边界条件)
4. **确定计算顺序**: 自底向上或自顶向下地计算所有状态
5. **构造最优解**: 通过记录的信息重建最优解

TIP

判断一个问题是否适合用动态规划解决: 看它是否具有最优子结构和重叠子问题两个特征。

2 线性 DP

线性 DP 是动态规划中最基础的一类, 具有以下特点:

- 状态转移只依赖前面的有限状态 (通常为前一个状态)
- 问题状态可以表示为一维数组
- 时间复杂度通常为 $O(n)$ 的问题

2.1 经典问题

I 最长递增子序列 (LIS)

最长递增子序列 (Longest Increasing Subsequence) 问题是指在一个给定的整数序列 `nums` 中, 找到最长的严格递增子序列的长度。

状态定义: 设 `dp[i]` 表示以 `nums[i]` 结尾的最长递增子序列的长度。

状态转移方程: 对于每个 `i`, 遍历 `j` 从 `0` 到 `i-1`: 如果 `nums[j] < nums[i]`, 则更新 `dp[i]`: $dp[i] = \max(dp[i], dp[j] + 1)$

初始条件: 每个元素自身可以作为长度为 1 的递增子序列, 因此 `dp[i] = 1` 对所有 `i`。

```
1  int lengthOfLIS(vector<int>& nums) {  
2      if (nums.empty()) return 0;  
3      int n = nums.size();  
4      vector<int> dp(n, 1); // 初始化 dp 数组, 长度为 1  
5  
6      for (int i = 1; i < n; ++i) {  
7          for (int j = 0; j < i; ++j) {  
8              if (nums[j] < nums[i]) {  
9                  dp[i] = max(dp[i], dp[j] + 1);  
10             }  
11         }  
12     }  
13  
14     return *max_element(dp.begin(), dp.end()); // 返回最长递增子序列的长度  
15 }
```

NOTE

该算法的时间复杂度为 $O(n^2)$ 。可以使用二分查找优化到 $O(n \log n)$ 。

I 最大子数组和 (Kadane's Algorithm)

最大子数组和 (Maximum Subarray Sum) 问题是指在一个给定的整数数组 `nums` 中, 找到一个连续子数组 (至少包含一个元素), 使得该子数组的和最大。

状态定义: 设 `dp[i]` 表示以 `nums[i]` 结尾的最大子数组和。

状态转移方程: $dp[i] = \max(nums[i], dp[i - 1] + nums[i])$, 即要么选择当前元素作为新的子数组开始, 要么将当前元素加入到之前的子数组中。

初始条件: $dp[0] = nums[0]$ 。

```
1  int maxSubArray(vector<int>& nums) {  
2      if (nums.empty()) return 0;  
3      int n = nums.size();  
4      vector<int> dp(n);  
5      dp[0] = nums[0];  
6      int maxSum = dp[0];  
7  
8      for (int i = 1; i < n; ++i) {  
9          dp[i] = max(nums[i], dp[i - 1] + nums[i]);  
10         maxSum = max(maxSum, dp[i]);  
11     }
```



```
12
13     return maxSum; // 返回最大子数组和
14 }
```

TIP

Kadane 算法可以进一步优化空间复杂度到 $O(1)$ ，只需要维护一个变量即可。

打家劫舍 (House Robber)

打家劫舍 (House Robber) 问题是指在一个给定的整数数组 `nums` 中，表示每个房屋中的金额，要求在不触动相邻房屋警报器的情况下，计算能够抢劫到的最高金额。

状态定义：设 `dp[i]` 表示抢劫到第 `i` 个房屋时能够获得最高金额。

状态转移方程： $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$ ，即要么不抢第 `i` 个房屋，金额为 `dp[i-1]`，要么抢第 `i` 个房屋，金额为 `dp[i-2] + nums[i]`。

初始条件： $dp[0] = nums[0]$ ， $dp[1] = \max(nums[0], nums[1])$ 。

```
1  int rob(vector<int>& nums) {
2      if (nums.empty()) return 0;
3      int n = nums.size();
4      if (n == 1) return nums[0];
5
6      vector<int> dp(n);
7      dp[0] = nums[0];
8      dp[1] = max(nums[0], nums[1]);
9
10     for (int i = 2; i < n; ++i) {
11         dp[i] = max(dp[i-1], dp[i-2] + nums[i]);
12     }
13
14     return dp[n-1]; // 返回最高金额
15 }
```

斐波那契数列

计算第 n 个斐波那契数： $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$

自顶向下 (记忆化递归)

```
1  // 记忆化数组
2  private static Map<Integer, Integer> memo = new HashMap<>();
3
4  public static int fibonacci(int n) {
5      // 边界条件
6      if (n <= 1) {
7          return n;
8      }
9
10     // 如果已经计算过，则直接返回
11     if (memo.containsKey(n)) {
12         return memo.get(n);
```

```
13     }
14
15     // 计算并存储结果
16     int result = fibonacci(n - 1) + fibonacci(n - 2);
17     memo.put(n, result);
18     return result;
19 }
```

- 时间复杂度: $O(n)$ (因为每个值只计算一次)
- 空间复杂度: $O(n)$ (用于存储递归调用栈和记忆化数组)

自底向上 (迭代)

```
1 public static int fibonacci(int n) { Java
2     // 边界条件
3     if (n <= 1) {
4         return n;
5     }
6
7     // 定义数组存储计算结果
8     int[] dp = new int[n + 1];
9     dp[0] = 0;
10    dp[1] = 1;
11
12    // 从第 2 个数开始迭代计算
13    for (int i = 2; i <= n; i++) {
14        dp[i] = dp[i - 1] + dp[i - 2];
15    }
16
17    // 返回第 n 个 Fibonacci 数
18    return dp[n];
19 }
```

- 时间复杂度: $O(n)$ (线性计算)
- 空间复杂度: $O(n)$ (用于存储动态规划数组)

空间优化

```
1 public static int fibonacci(int n) { Java
2     // 边界条件
3     if (n <= 1) {
4         return n;
5     }
6
7     // 只保存最近两个数
8     int prev1 = 0; // F(n-2)
9     int prev2 = 1; // F(n-1)
10
11    // 迭代计算
12    for (int i = 2; i <= n; i++) {
13        int current = prev1 + prev2; // 当前 Fibonacci 数
14        prev1 = prev2; // 更新 F(n-2)
15        prev2 = current; // 更新 F(n-1)
16    }
```

```
17
18    // 返回第 n 个 Fibonacci 数
19    return prev2;
20 }
```

- 时间复杂度: $O(n)$ (线性计算)
- 空间复杂度: $O(1)$ (只存储两个数)

NOTE

斐波那契数列是理解动态规划的经典入门案例, 展示了从递归到记忆化再到迭代的优化过程。

3 背包问题

背包问题是动态规划中的经典问题类型, 具有重要的理论和实践价值。

3.1 0-1 背包问题

给定一个容量为 W 的背包和 n 件物品, 每件物品有重量 $wt[i]$ 和价值 $val[i]$, 我们需要选择一些物品装入背包, 使得总价值最大化, 同时总重量不超过 W 。

状态分析

状态是动态规划中用来描述子问题的变量。分析状态需要明确:

- 问题的规模: 选择前 i 个物品时, 当前的背包容量为 w
- 状态的定义: 用 $dp[i][w]$ 表示前 i 件物品在背包容量 w 下的最大价值
- 状态的范围:
 - i 的范围是从 0 到 n
 - w 的范围是从 0 到 W

状态转移方程

状态转移方程用于描述如何从一个状态转移到另一个状态。它的确定过程如下:

决策分析: 对于第 i 件物品, 我们有两种决策:

1. 不选择第 i 件物品: 当前的最大价值是不放第 i 件物品的情况, 即: $dp[i][w] = dp[i-1][w]$
2. 选择第 i 件物品: 当前的最大价值是放第 i 件物品的情况。此时剩余背包容量为 $w-wt[i-1]$, 背包中的价值增加了 $val[i-1]$, 即: $dp[i][w] = dp[i-1][w-wt[i-1]] + val[i-1]$

选择最大值: 在两种决策中, 我们选择价值更大的决策:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i-1]] + val[i-1])$$

边界条件:

- 如果没有物品 (即 $i = 0$), 最大价值为 0, 即: $dp[0][w] = 0 \quad \forall w$
- 如果背包容量为 0 (即 $w = 0$), 最大价值也为 0, 即: $dp[i][0] = 0 \quad \forall i$

二维数组实现

```
1 public static int knapsack(int[] wt, int[] val, int W) {
```

 Java

```
2     int n = wt.length;
3     int[][] dp = new int[n + 1][W + 1];
4
5     for (int i = 1; i <= n; i++) {
6         for (int w = 1; w <= W; w++) {
7             if (wt[i - 1] <= w) {
8                 dp[i][w] = Math.max(dp[i - 1][w], dp[i - 1][w - wt[i - 1]] + val[i - 1]);
9             } else {
10                dp[i][w] = dp[i - 1][w];
11            }
12        }
13    }
14
15    return dp[n][W];
16 }
```

一维数组优化（空间优化）

在常规的二维数组 `dp[i][w]` 中，当前状态只依赖上一行的状态，因此可以优化为一维数组

`dp[w]`：从容量 W 到 0 逆序更新，避免覆盖上一行的数据。

```
1 public static int knapsackOptimized(int[] wt, int[] val, int W) {
2     int n = wt.length;
3     int[] dp = new int[W + 1];
4
5     for (int i = 0; i < n; i++) {
6         for (int w = W; w >= wt[i]; w--) {
7             dp[w] = Math.max(dp[w], dp[w - wt[i]] + val[i]);
8         }
9     }
10
11    return dp[W];
12 }
```

TIP

0-1 背包的一维优化关键在于逆序遍历容量，这样可以保证每个物品只被选择一次。

完全背包与多重背包

完全背包：每种物品有无限个可用

- 状态转移方程类似，但遍历容量时需要正序遍历
- 代码只需将内层循环改为：`for (int w = wt[i]; w <= W; w++)`

多重背包：每种物品有有限个（数量为 `count[i]`）

- 可以将多重背包转化为 0-1 背包（二进制优化）
- 或者使用单调队列优化到 $O(nW)$

CAUTION

完全背包和 0-1 背包的唯一区别在于遍历顺序：完全背包正序，0-1 背包逆序。这是一个容易混淆的点。

4 区间 DP

区间 DP 是动态规划的一种重要类型，专门解决涉及区间操作的问题。

4.1 核心思想

区间 DP 的核心思想包括：

- 状态定义： $dp[i][j]$ 表示区间 $[i, j]$ 上的最优解
- 区间划分：将大区间划分为两个或多个小区间求解
- 转移策略：通过枚举分割点 k 将区间分为 $[i, k]$ 和 $[k + 1, j]$
- 合并子解：将小区间的最优解合并得到大区间的解

4.2 经典问题

石子合并问题

有 N 堆石子排成一排，每堆石子有一定的数量。现要将 N 堆石子并成一堆。合并的过程只能每次将相邻的两堆石子堆成一堆，每次合并花费的代价为这两堆石子的和，经过 $N-1$ 次合并后成为一堆。求出总的代价最小值。

状态定义： $dp[i][j]$ 表示合并第 i 堆到第 j 堆石子的最小代价

状态转移方程： $dp[i][j] = \min(dp[i][k] + dp[k + 1][j] + sum[i][j])$ ，其中 $i \leq k < j$ ， $sum[i][j]$ 表示第 i 堆到第 j 堆石子的总和

初始条件： $dp[i][i] = 0$ （单堆石子不需要合并）

计算顺序：按区间长度从小到大计算

```
1  int mergeStones(vector<int>& stones) {
2      int n = stones.size();
3      vector<vector<int>> dp(n, vector<int>(n, INT_MAX));
4      vector<int> prefixSum(n + 1, 0);
5
6      // 计算前缀和
7      for (int i = 0; i < n; ++i) {
8          prefixSum[i + 1] = prefixSum[i] + stones[i];
9      }
10
11     // 初始化：单堆石子代价为0
12     for (int i = 0; i < n; ++i) {
13         dp[i][i] = 0;
14     }
15
16     // 按区间长度从小到大计算
17     for (int len = 2; len <= n; ++len) {
18         for (int i = 0; i <= n - len; ++i) {
19             int j = i + len - 1;
20             int sum = prefixSum[j + 1] - prefixSum[i];
21
22             for (int k = i; k < j; ++k) {
```

```

23         dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j] + sum);
24     }
25 }
26 }
27
28 return dp[0][n - 1];
29 }

```

NOTE

石子合并问题的时间复杂度为 $O(n^3)$ ，可以通过四边形不等式优化到 $O(n^2)$ 。

I 最长回文子序列

给定一个字符串 s ，找到其中最长的回文子序列的长度。

状态定义： $dp[i][j]$ 表示字符串 s 中从位置 i 到位置 j 的最长回文子序列长度

状态转移方程：

- 如果 $s[i] == s[j]$: $dp[i][j] = dp[i + 1][j - 1] + 2$
- 如果 $s[i] \neq s[j]$: $dp[i][j] = \max(dp[i + 1][j], dp[i][j - 1])$

初始条件： $dp[i][i] = 1$ (单个字符是回文)

```

1  int longestPalindromeSubseq(string s) {
2      int n = s.size();
3      vector<vector<int>> dp(n, vector<int>(n, 0));
4
5      // 单个字符的回文长度为1
6      for (int i = 0; i < n; ++i) {
7          dp[i][i] = 1;
8      }
9
10     // 按区间长度从小到大计算
11     for (int len = 2; len <= n; ++len) {
12         for (int i = 0; i <= n - len; ++i) {
13             int j = i + len - 1;
14             if (s[i] == s[j]) {
15                 dp[i][j] = dp[i + 1][j - 1] + 2;
16             } else {
17                 dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
18             }
19         }
20     }
21
22     return dp[0][n - 1];
23 }

```

4.3 优化技巧

I 四边形不等式优化

对于某些区间 DP 问题，如果满足四边形不等式条件，可以将时间复杂度从 $O(n^3)$ 优化到 $O(n^2)$ 。

四边形不等式：如果函数 $w(i, j)$ 满足：

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c) \quad (a \leq b \leq c \leq d)$$

则可以使用四边形不等式优化。

TIP

四边形不等式优化的关键是证明决策单调性，即最优分割点随区间右端点增加而单调不减。

环状结构处理（破坏成链）

对于环形结构的区间 DP 问题，常用的技巧是**破坏成链**：

- 将原数组复制一份接到末尾，形成长度为 $2n$ 的数组
- 在这个扩展的数组上进行区间 DP
- 最终答案在所有长度为 n 的区间中取最优值

5 树形 DP

树形动态规划是一种在树结构上进行的动态规划算法，它利用树的递归特性，通过深度优先遍历（DFS）自底向上或自顶向下地求解问题。

5.1 特点

树形 DP 具有以下特点：

- **树形结构：**问题基于树或森林结构（无向无环图）
- **递归性质：**问题可以分解为子树上的子问题
- **最优子结构：**整体最优解包含子树最优解
- **无后效性：**当前决策只影响子树，不影响祖先节点

5.2 经典问题

树的直径

树的直径是指树上任意两点间的最长路径。

解题思路：

- 对每个节点，计算经过该节点的最长路径
- 最长路径 = 该节点到其子树中最深节点的距离 + 次深节点的距离
- 树的直径是所有节点中最长路径的最大值

```
1  int diameter = 0;
2
3  int dfs(TreeNode* node) {
4      if (!node) return 0;
5
6      int leftDepth = dfs(node->left);
7      int rightDepth = dfs(node->right);
8
9      // 更新直径
10     diameter = max(diameter, leftDepth + rightDepth);
```



```

11
12     // 返回当前节点的深度
13     return max(leftDepth, rightDepth) + 1;
14 }
15
16 int diameterOfBinaryTree(TreeNode* root) {
17     dfs(root);
18     return diameter;
19 }

```

没有上司的舞会（最大独立集）

在一棵树中，选择一些节点使得任意两个被选节点不相邻，求选中节点权值和的最大值。

状态定义：

- $dp[u][0]$ ：不选择节点 u 时，以 u 为根的子树的最大权值和
- $dp[u][1]$ ：选择节点 u 时，以 u 为根的子树的最大权值和

状态转移方程：

- $dp[u][0] = \sum(\max(dp[v][0], dp[v][1]))$ ，其中 v 是 u 的子节点
- $dp[u][1] = weight[u] + \sum(dp[v][0])$

最终答案： $\max(dp[root][0], dp[root][1])$

```

1  vector<vector<int>> adj;
2  vector<int> weight;
3  vector<vector<int>> dp;
4
5  void dfs(int u, int parent) {
6      dp[u][0] = 0;
7      dp[u][1] = weight[u];
8
9      for (int v : adj[u]) {
10         if (v == parent) continue;
11         dfs(v, u);
12
13         dp[u][0] += max(dp[v][0], dp[v][1]);
14         dp[u][1] += dp[v][0];
15     }
16 }
17
18 int maxIndependentSet(int n) {
19     dp.resize(n, vector<int>(2, 0));
20     dfs(0, -1);
21     return max(dp[0][0], dp[0][1]);
22 }

```

NOTE

树形 DP 通常使用后序遍历（先处理子节点，再处理父节点），这保证了计算父节点时子节点的状态已经确定。

5.3 高级技巧与优化

I 换根 DP (二次扫描)

换根 DP 用于解决以每个节点为根时的相关问题。基本思路：

1. 第一次 DFS：任选一个根，计算以该根为根的树的相关信息
2. 第二次 DFS：通过换根操作，快速计算以其他节点为根时的信息

应用场景：

- 树的重心
- 树上所有节点到其他节点的距离和
- 树的直径端点

TIP

换根 DP 的关键在于推导从一个根转移到相邻节点根时的状态变化公式。

I 虚树优化

当树的节点很多，但只有少数关键点需要处理时，可以构建虚树：

- 只保留关键点和它们的最近公共祖先（LCA）
- 在虚树上进行 DP，大幅减少计算量
- 时间复杂度从 $O(n)$ 降低到 $O(k \log k)$ ，其中 k 是关键点数量

6 状态压缩 DP

状态压缩动态规划是一种高效处理组合优化问题的技术，特别适用于状态空间呈指数级增长的问题。

6.1 基本概念

状态压缩 DP 的核心思想：

- 状态压缩：使用整数的二进制位表示状态，每个位对应一个布尔值（0/1）
- 位运算：使用位操作快速进行状态转移和状态检查
- 状态空间优化：将指数级状态空间压缩到有限整数范围内

使用场景：

- 状态维度高但每个维度取值有限（通常为布尔值）
- 状态空间可达 2^{20} （约 100 万）以内
- 问题具有最优子结构和无后效性

核心技巧：

- 状态表示： $dp[state][*]$ ，其中 $state$ 是整数表示的压缩状态
- 位运算操作：
 - 检查状态：`state & (1 << i)`
 - 设置状态：`state | (1 << i)`
 - 清除状态：`state & ~(1 << i)`
 - 状态交集：`state1 & state2`
 - 状态并集：`state1 | state2`

6.2 经典问题

I 旅行商问题 (TSP)

旅行商问题：给定 n 个城市和城市之间的距离，求从起点出发，访问所有城市恰好一次并回到起点的最短路径。

状态定义： $dp[mask][i]$ 表示已访问城市集合为 $mask$ ，当前在城市 i 的最短路径长度

状态转移方程： $dp[mask][i] = \min(dp[mask \oplus (1 \ll i)][j] + dist[j][i])$ ，其中 j 是 $mask$ 中除 i 外的城市

初始条件： $dp[1][0] = 0$ （从城市 0 出发）

最终答案： $\min(dp[(1 \ll n) - 1][i] + dist[i][0])$ ，其中 $i \neq 0$

```

1  int tsp(vector<vector<int>>& dist) {
2      int n = dist.size();
3      int fullMask = (1 << n) - 1;
4      vector<vector<int>> dp(fullMask + 1, vector<int>(n, INT_MAX));
5
6      // 初始状态：从城市0出发
7      dp[1][0] = 0;
8
9      // 枚举所有状态
10     for (int mask = 1; mask <= fullMask; ++mask) {
11         for (int i = 0; i < n; ++i) {
12             if (!(mask & (1 << i))) continue; // 城市i不在mask中
13             if (dp[mask][i] == INT_MAX) continue;
14
15             // 尝试前往未访问的城市
16             for (int j = 0; j < n; ++j) {
17                 if (mask & (1 << j)) continue; // 城市j已访问
18                 int newMask = mask | (1 << j);
19                 dp[newMask][j] = min(dp[newMask][j], dp[mask][i] + dist[i][j]);
20             }
21         }
22     }
23
24     // 回到起点
25     int ans = INT_MAX;
26     for (int i = 1; i < n; ++i) {
27         if (dp[fullMask][i] != INT_MAX) {
28             ans = min(ans, dp[fullMask][i] + dist[i][0]);
29         }
30     }
31
32     return ans;
33 }

```

CAUTION

TSP 问题的时间复杂度为 $O(2^n \cdot n^2)$ ，适用于 $n \leq 20$ 的小规模问题。

I 棋盘覆盖问题 (多米诺骨牌)

用 1×2 的多米诺骨牌覆盖 $n \times m$ 的棋盘，求方案数。

状态定义: $dp[i][mask]$ 表示处理到第 i 行, 当前行的覆盖状态为 $mask$ 的方案数

状态转移: 枚举下一行的所有合法状态, 进行转移

NOTE

棋盘覆盖问题通常需要预处理所有合法状态和状态之间的转移关系。

6.3 优化技巧

滚动数组优化

如果状态转移只依赖于前一层的状态, 可以使用滚动数组将空间复杂度从 $O(2^n)$ 降低到 $O(2^{n-1})$ 。

状态子集枚举优化

对于某些问题, 可以只枚举状态的子集, 而不是所有状态, 从而减少计算量。

子集枚举技巧:

```
1 // 枚举 mask 的所有子集
2 for (int sub = mask; sub; sub = (sub - 1) & mask) {
3     // 处理子集 sub
4 }
```

**TIP**

子集枚举的时间复杂度为 $O(3^n)$, 比枚举所有状态对的 $O(4^n)$ 更优。

7 数位 DP

数位动态规划是解决与数字的数位相关问题的强大技术, 特别适用于统计满足特定条件的数字数量、计算数字和等场景。

7.1 基本概念

数位 DP 结合了动态规划和深度优先搜索的思想:

- 数位处理: 将数字按位分解 (十进制或二进制)
- 状态压缩: 记录当前处理位置、是否受限、前导零状态等信息
- 记忆化搜索: 存储已计算状态避免重复计算

适用场景:

- 统计区间 $[A, B]$ 内满足特定条件的数字数量
- 计算数字的数位属性 (和、乘积、特定模式等)
- 处理超大范围数字 (10^{18} 级别)

关键技巧:

- 状态表示: $dp[pos][tight][leading_zero][state]$
 - pos : 当前处理的位置 (从高位到低位)
 - $tight$: 是否受限 (当前位能否任意取值)
 - $leading_zero$: 是否存在前导零
 - $state$: 问题特定状态 (如前几位值、和、乘积等)

- 位运算：高效处理二进制相关问题
- 前导零处理：区分真实 0 和前导 0

7.2 经典问题

I 统计不含连续 1 的二进制数

统计 $[0, n]$ 范围内，二进制表示中不含连续 1 的数字个数。

状态定义： $dp[pos][prev_one][tight]$

- pos ：当前处理的位
- $prev_one$ ：前一位是否为 1
- $tight$ ：是否受限于 n 的对应位

状态转移：

- 如果前一位是 1，当前位只能填 0
- 如果前一位是 0，当前位可以填 0 或 1（需考虑 $tight$ 限制）

```

1  vector<vector<vector<int>>> memo;
2  string binary;
3
4  int dfs(int pos, bool prevOne, bool tight) {
5      if (pos == binary.size()) return 1;
6      if (!tight && memo[pos][prevOne][0] != -1)
7          return memo[pos][prevOne][0];
8      if (tight && memo[pos][prevOne][1] != -1)
9          return memo[pos][prevOne][1];
10
11     int limit = tight ? (binary[pos] - '0') : 1;
12     int result = 0;
13
14     for (int digit = 0; digit <= limit; ++digit) {
15         if (prevOne && digit == 1) continue; // 不能有连续的1
16         result += dfs(pos + 1, digit == 1, tight && (digit == limit));
17     }
18
19     if (!tight) memo[pos][prevOne][0] = result;
20     else memo[pos][prevOne][1] = result;
21
22     return result;
23 }
24
25 int countNumbers(int n) {
26     // 转换为二进制
27     binary = "";
28     while (n > 0) {
29         binary = char('0' + (n % 2)) + binary;
30         n /= 2;
31     }
32     if (binary.empty()) binary = "0";
33
34     memo.assign(binary.size(), vector<vector<int>>(2, vector<int>(2, -1)));

```

```
35     return dfs(0, false, true);  
36 }
```

统计区间内数位和等于 K 的数字个数

统计 $[A, B]$ 范围内，数位和等于 K 的数字个数。

解题思路：

- 转化为求 $[0, B]$ 和 $[0, A - 1]$ 的结果之差
- 状态： $dp[pos][sum][tight]$
- 转移：枚举当前位的数字（0-9），累加到 sum

NOTE

数位 DP 通常使用记忆化搜索实现，需要注意处理前导零和边界条件。

8 DP 优化技巧总结

8.1 空间优化

滚动数组

当状态转移只依赖于前一层或前几个状态时，可以使用滚动数组减少空间复杂度。

示例：0-1 背包从 $O(nW)$ 优化到 $O(W)$

8.2 时间优化

单调队列优化

适用于状态转移方程形如： $dp[i] = \frac{\min}{\max(dp[j] + f(i, j))}$ 的问题。

核心思想：维护一个单调队列，快速找到最优的 j。

时间复杂度：从 $O(n^2)$ 优化到 $O(n)$

斜率优化

适用于状态转移方程可以转化为斜率形式的问题。

核心思想：将状态看作平面上的点，通过维护凸包快速找到最优决策点。

应用场景：

- 任务调度问题
- 仓库建设问题

决策单调性分治优化

如果决策点具有单调性（即 $opt[i] \leq opt[i + 1]$ ），可以使用分治优化。

核心思想：

- 对于区间 $[l, r]$ ，已知最优决策点在 $[optL, optR]$ 范围内

- 计算中间位置 `mid` 的最优决策点 `optMid`
- 递归处理 `[l, mid - 1]` 和 `[mid + 1, r]`

时间复杂度：从 $O(n^2)$ 优化到 $O(n \log n)$

TIP

掌握这些优化技巧需要在大量练习中积累经验，建议先从基础的线性 DP 和背包问题入手，逐步挑战更复杂的优化问题。

9 线性 DP 补充

9.1 最长公共子序列 (LCS)

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长公共子序列的长度。

状态定义： $dp[i][j]$ 表示 `text1` 的前 i 个字符和 `text2` 的前 j 个字符的最长公共子序列长度

状态转移方程：

- 如果 `text1[i - 1] == text2[j - 1]`： $dp[i][j] = dp[i - 1][j - 1] + 1$
- 否则： $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$

初始条件： $dp[0][j] = 0$, $dp[i][0] = 0$

```

1  int longestCommonSubsequence(string text1, string text2) {
2      int m = text1.size(), n = text2.size();
3      vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
4
5      for (int i = 1; i <= m; ++i) {
6          for (int j = 1; j <= n; ++j) {
7              if (text1[i - 1] == text2[j - 1]) {
8                  dp[i][j] = dp[i - 1][j - 1] + 1;
9              } else {
10                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
11             }
12         }
13     }
14
15     return dp[m][n];
16 }

```

NOTE

LCS 问题是经典的二维 DP 问题，时间复杂度为 $O(mn)$ 。

9.2 编辑距离 (Edit Distance)

给定两个单词 `word1` 和 `word2`，计算出将 `word1` 转换成 `word2` 所使用的最少操作数。可以进行的操作有：插入一个字符、删除一个字符、替换一个字符。

状态定义： $dp[i][j]$ 表示将 `word1` 的前 i 个字符转换为 `word2` 的前 j 个字符所需的最少操作数

状态转移方程：

- 如果 $word1[i-1] == word2[j-1]$: $dp[i][j] = dp[i-1][j-1]$
- 否则: $dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + 1)$
 - $dp[i-1][j] + 1$: 删除操作
 - $dp[i][j-1] + 1$: 插入操作
 - $dp[i-1][j-1] + 1$: 替换操作

初始条件: $dp[i][0] = i$, $dp[0][j] = j$

```

1  int minDistance(string word1, string word2) {
2      int m = word1.size(), n = word2.size();
3      vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
4
5      // 初始化边界
6      for (int i = 0; i <= m; ++i) dp[i][0] = i;
7      for (int j = 0; j <= n; ++j) dp[0][j] = j;
8
9      for (int i = 1; i <= m; ++i) {
10         for (int j = 1; j <= n; ++j) {
11             if (word1[i - 1] == word2[j - 1]) {
12                 dp[i][j] = dp[i - 1][j - 1];
13             } else {
14                 dp[i][j] = min({dp[i - 1][j] + 1,      // 删除
15                                dp[i][j - 1] + 1,      // 插入
16                                dp[i - 1][j - 1] + 1}); // 替换
17             }
18         }
19     }
20
21     return dp[m][n];
22 }

```

TIP

编辑距离是自然语言处理中的重要算法，用于拼写检查、DNA 序列比对等场景。

9.3 买股票的最佳时机系列

买卖股票的最佳时机 I（只能交易一次）

状态定义：

- $dp[i][0]$: 第 i 天不持有股票的最大利润
- $dp[i][1]$: 第 i 天持有股票的最大利润

状态转移方程：

- $dp[i][0] = \max(dp[i-1][0], dp[i-1][1] + prices[i])$
- $dp[i][1] = \max(dp[i-1][1], -prices[i])$

```

1  int maxProfit(vector<int>& prices) {
2      int n = prices.size();
3      if (n == 0) return 0;
4
5      vector<vector<int>> dp(n, vector<int>(2, 0));

```

```

6    dp[0][0] = 0;
7    dp[0][1] = -prices[0];
8
9    for (int i = 1; i < n; ++i) {
10       dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] + prices[i]);
11       dp[i][1] = max(dp[i - 1][1], -prices[i]);
12    }
13
14    return dp[n - 1][0];
15 }

```

买卖股票的最佳时机 II (无限次交易)

状态转移方程：

- $dp[i][0] = \max(dp[i - 1][0], dp[i - 1][1] + prices[i])$
- $dp[i][1] = \max(dp[i - 1][1], dp[i - 1][0] - prices[i])$

NOTE

与版本 I 的区别在于：买入时可以从之前的利润中扣除，而不是从 0 开始。

10 概率 DP 与期望 DP

概率 DP 和期望 DP 是动态规划的重要分支，用于解决涉及随机性的问题。

10.1 基本概念

概率 DP：计算达到某种状态的概率

期望 DP：计算达到某种状态的期望值（通常是步数、代价等）

核心公式：

- 期望的线性性： $E[X + Y] = E[X] + E[Y]$
- 全期望公式： $E[X] = \sum (E[X|A_i] \cdot P(A_i))$

10.2 经典问题

掷骰子问题

有一个 n 面的骰子，问掷出所有面的期望次数是多少？

状态定义： $dp[i]$ 表示已经掷出 i 个不同面，还需要掷多少次才能掷出所有 n 个面的期望

状态转移方程：

$$dp[i] = \frac{i}{n} \cdot dp[i] + \frac{n-i}{n} \cdot dp[i+1] + 1$$

$$\text{化简得: } dp[i] = dp[i+1] + \frac{n-i}{n}$$

初始条件： $dp[n] = 0$

最终答案： $dp[0]$

```
1 double expectedRolls(int n) {
```

 C++


```
2    vector<double> dp(n + 1, 0);
3    dp[n] = 0;
4
5    for (int i = n - 1; i >= 0; --i) {
6        dp[i] = dp[i + 1] + (double)n / (n - i);
7    }
8
9    return dp[0];
10 }
```

NOTE

这个问题的答案是调和级数： $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$

收集卡片问题

有 n 种卡片，每次随机获得一种，问收集齐所有卡片的期望次数。

这个问题与掷骰子问题本质相同，答案也是 $n \cdot H_n$ 。

10.3 解题技巧

逆向思维：期望 DP 通常从目标状态逆向推导到初始状态

状态设计：

- 概率 DP：状态通常表示“已经达到某种条件”
- 期望 DP：状态通常表示“从当前状态到目标状态的期望步数”

边界处理：

- 概率 DP：初始状态概率为 1，其他为 0
- 期望 DP：目标状态的期望为 0

CAUTION

期望 DP 容易出现循环依赖，需要仔细设计状态转移顺序，或者使用高斯消元求解方程组。

11 DP 建模思路总结

11.1 如何判断一个问题是否可以用 DP 解决

关键特征

- **最优子结构：**问题的最优解包含子问题的最优解
- **重叠子问题：**相同的子问题会被多次计算
- **无后效性：**未来的决策只与当前状态有关，与如何到达当前状态无关

识别信号

- 问题要求“最大值”、“最小值”、“方案数”
- 问题涉及“选择”或“不选择”

- 问题可以分解为规模更小的子问题
- 暴力递归存在大量重复计算

11.2 DP 问题建模步骤

第一步：确定状态

思考方式：

- 问题的变量有哪些？
- 哪些变量会影响最终结果？
- 如何用这些变量唯一标识一个子问题？

常见状态类型：

- 位置/索引： $dp[i]$ 、 $dp[i][j]$
- 集合： $dp[mask]$ （状压 DP）
- 剩余容量： $dp[i][w]$ （背包）
- 前缀/后缀： $dp[i]$ 表示前 i 个元素

第二步：确定状态转移方程

思考方式：

- 当前状态可以由哪些状态转移而来？
- 每种转移的代价/收益是什么？
- 如何选择最优的转移？

常见转移类型：

- 线性转移： $dp[i]$ 从 $dp[i-1]$ 、 $dp[i-2]$ 等转移
- 区间转移： $dp[i][j]$ 从 $dp[i][k]$ 和 $dp[k+1][j]$ 转移
- 集合转移： $dp[mask]$ 从 $dp[mask \oplus (1 \ll i)]$ 转移

第三步：确定初始条件

思考方式：

- 最小子问题的解是什么？
- 边界情况如何处理？

常见初始条件：

- $dp[0] = 0$ 或 $dp[0] = 1$ （取决于问题）
- $dp[i][i] = 0$ （区间 DP）
- $dp[0][w] = 0$ （背包问题）

第四步：确定计算顺序

思考方式：

- 计算 $dp[i]$ 时，它依赖的状态是否已经计算过？
- 应该从小到大还是从大到小枚举？

常见顺序：

- 线性 DP：从小到大枚举 i
- 区间 DP：按区间长度从小到大

- 树形 DP：后序遍历（先子节点后父节点）
- 状压 DP：按 mask 从小到大

11.3 常见错误与调试技巧

常见错误

- 状态定义不清：导致转移方程难以推导
- 边界条件错误：特别是 $dp[0]$ 的值
- 转移顺序错误：依赖的状态还未计算
- 数组越界：访问 $dp[-1]$ 或 $dp[n]$
- 初始化遗漏：某些状态未正确初始化

调试技巧

- 打印 DP 表：观察中间状态是否符合预期
- 小规模测试：用手工计算验证小数据的结果
- 对比暴力解：用小数据对比 DP 和暴力解的结果
- 检查边界：特别关注 $i = 0$ 、 $i = n - 1$ 等边界情况

TIP

DP 问题的难点不在于代码实现，而在于正确的状态定义和转移方程设计。多做题、多总结是提高 DP 能力的关键。

12 DP 问题分类速查

12.1 按问题类型分类

类型	典型问题	状态维度
线性 DP	LIS、LCS、编辑距离	1D 或 2D
背包 DP	0-1 背包、完全背包	2D
区间 DP	石子合并、矩阵连乘	2D
树形 DP	树的直径、最大独立集	树上 DFS
状压 DP	TSP、棋盘覆盖	mask + 其他
数位 DP	数字计数、数位统计	pos + tight
概率 DP	期望步数、概率计算	视问题而定

12.2 按优化技巧分类

优化方法	适用场景	复杂度改进
滚动数组	只依赖前一层	$O(n) \rightarrow O(1)$ 空间
单调队列	滑动窗口最值	$O(n^2) \rightarrow O(n)$

斜率优化	特定形式的转移	$O(n^2) \rightarrow O(n)$
四边形不等式	区间 DP	$O(n^3) \rightarrow O(n^2)$
决策单调性	决策点单调	$O(n^2) \rightarrow O(n \log n)$



本章完

Chapter 18

回溯法与分支限界

NOTE

回溯法和分支限界都是系统搜索解空间的方法，用于解决组合优化问题。

1 回溯法的基本思想

回溯法是一种系统性搜索解空间的方法，通过深度优先搜索和剪枝来寻找问题的解。

1.1 解空间树

回溯法将问题的解空间组织成树形结构。

子集树

当问题是从 n 个元素中选择子集时，解空间是子集树。

特点：

- 树的高度为 n
- 每个节点有两个分支（选/不选）
- 叶子节点数为 2^n

示例：0/1 背包、子集和问题

排列树

当问题是求 n 个元素的排列时，解空间是排列树。

特点：

- 树的高度为 n
- 第 i 层有 $n - i + 1$ 个分支
- 叶子节点数为 $n!$

示例：旅行商问题、全排列

1.2 约束函数与限界函数

约束函数（Constraint Function）

判断当前部分解是否满足问题的约束条件。

作用：剪去不满足约束的子树

示例：N 皇后问题中，检查新放置的皇后是否与已有皇后冲突

| 限界函数 (Bound Function)

判断当前部分解是否可能达到**最优解**。

作用：剪去不可能产生更优解的子树

示例：0/1 背包中，如果当前价值 + 剩余最大可能价值 < 已知最优值，则剪枝

2 回溯法的框架与剪枝技巧

2.1 回溯法通用框架

```
1 void backtrack(int level) {  
2     if (level > n) {  
3         // 找到完整解，处理结果  
4         processSolution();  
5         return;  
6     }  
7  
8     for (每个可能的选择) {  
9         if (约束函数(level, 选择)) { // 剪枝  
10            做出选择;  
11            backtrack(level + 1);  
12            撤销选择; // 回溯  
13        }  
14    }  
15 }
```

2.2 剪枝技巧

| 可行性剪枝

提前判断当前路径是否可行。

示例：N 皇后中，检查新位置是否与已有皇后冲突

| 最优性剪枝

如果当前解已经不如已知最优解，则剪枝。

示例：TSP 中，如果当前路径长度已经超过已知最短路径，则停止搜索

| 对称性剪枝

利用问题的对称性减少搜索空间。

示例：组合问题中，规定元素按顺序选择，避免重复

TIP

好的剪枝策略可以将指数级复杂度降低几个数量级。

3 经典回溯问题

3.1 N 皇后问题

问题：在 $n \times n$ 棋盘上放置 n 个皇后，使得它们互不攻击。

```
1  vector<vector<string>> solveNQueens(int n) {  
2      vector<vector<string>> results;  
3      vector<string> board(n, string(n, '.'));  
4      vector<bool> cols(n), diag1(2*n-1), diag2(2*n-1);  
5  
6      function<void(int)> backtrack = [&](int row) {  
7          if (row == n) {  
8              results.push_back(board);  
9              return;  
10         }  
11  
12         for (int col = 0; col < n; ++col) {  
13             int d1 = row - col + n - 1;  
14             int d2 = row + col;  
15  
16             if (cols[col] || diag1[d1] || diag2[d2]) continue;  
17  
18             // 放置皇后  
19             board[row][col] = 'Q';  
20             cols[col] = diag1[d1] = diag2[d2] = true;  
21  
22             backtrack(row + 1);  
23  
24             // 回溯  
25             board[row][col] = '.';  
26             cols[col] = diag1[d1] = diag2[d2] = false;  
27         }  
28     };  
29  
30     backtrack(0);  
31     return results;  
32 }
```

时间复杂度： $O(n!)$ （最坏情况）

3.2 数独求解

问题：填充 9×9 数独网格。

```
1  bool solveSudoku(vector<vector<char>>& board) {  
2      for (int i = 0; i < 9; ++i) {  
3          for (int j = 0; j < 9; ++j) {  
4              if (board[i][j] == '.') {  
5                  for (char c = '1'; c <= '9'; ++c) {  
6                      if (isValid(board, i, j, c)) {
```

```

7         board[i][j] = c;
8         if (solveSudoku(board)) return true;
9         board[i][j] = '.'; // 回溯
10    }
11    }
12    return false;
13    }
14    }
15    }
16    return true;
17    }

```

3.3 图的 m 着色问题

问题：用 m 种颜色给图的顶点着色，相邻顶点颜色不同。

```

1  bool graphColoring(vector<vector<int>>& graph, int m) {
2      int n = graph.size();
3      vector<int> color(n, 0);
4
5      function<bool(int)> backtrack = [&](int v) {
6          if (v == n) return true;
7
8          for (int c = 1; c <= m; ++c) {
9              if (isSafe(graph, color, v, c)) {
10                 color[v] = c;
11                 if (backtrack(v + 1)) return true;
12                 color[v] = 0; // 回溯
13             }
14         }
15         return false;
16     };
17
18     return backtrack(0);
19 }

```

4 子集与排列问题

4.1 子集和问题

问题：从数组中选择若干元素，使得它们的和等于目标值。

```

1  vector<vector<int>> subsetSum(vector<int>& nums, int target) {
2      vector<vector<int>> results;
3      vector<int> current;
4
5      function<void(int, int)> backtrack = [&](int start, int sum) {
6          if (sum == target) {
7              results.push_back(current);
8              return;

```



```
9     }
10
11     for (int i = start; i < nums.size(); ++i) {
12         if (sum + nums[i] > target) continue; // 剪枝
13
14         current.push_back(nums[i]);
15         backtrack(i + 1, sum + nums[i]);
16         current.pop_back(); // 回溯
17     }
18 };
19
20 sort(nums.begin(), nums.end());
21 backtrack(0, 0);
22 return results;
23 }
```

4.2 全排列

问题：生成数组的所有排列。

```
1  vector<vector<int>> permute(vector<int>& nums) { 
2      vector<vector<int>> results;
3      vector<bool> used(nums.size(), false);
4      vector<int> current;
5
6      function<void()> backtrack = [&]() {
7          if (current.size() == nums.size()) {
8              results.push_back(current);
9              return;
10         }
11
12         for (int i = 0; i < nums.size(); ++i) {
13             if (used[i]) continue;
14
15             used[i] = true;
16             current.push_back(nums[i]);
17             backtrack();
18             current.pop_back();
19             used[i] = false; // 回溯
20         }
21     };
22
23     backtrack();
24     return results;
25 }
```

4.3 组合问题

问题：从 n 个数中选择 k 个数的所有组合。

```
1  vector<vector<int>> combine(int n, int k) { 
2      vector<vector<int>> results;
```

```
3     vector<int> current;
4
5     function<void(int)> backtrack = [&](int start) {
6         if (current.size() == k) {
7             results.push_back(current);
8             return;
9         }
10
11        for (int i = start; i <= n; ++i) {
12            current.push_back(i);
13            backtrack(i + 1);
14            current.pop_back(); // 回溯
15        }
16    };
17
18    backtrack(1);
19    return results;
20 }
```

5 分支限界法的基本概念

分支限界法是另一种系统搜索解空间的方法，使用**广度优先**或**最佳优先**策略。

5.1 与回溯法的区别

特性	回溯法	分支限界
搜索策略	深度优先	广度优先/最佳优先
存储结构	栈（递归）	队列/优先队列
求解目标	所有解/一个解	最优解
剪枝方式	约束函数	约束+限界函数

5.2 FIFO 分支限界

使用**队列**实现**广度优先**搜索。

特点：

- 按层次扩展节点
- 保证找到最短路径（如果存在）
- 空间复杂度较高

5.3 优先队列式分支限界（LC 分支限界）

使用**优先队列**，每次扩展最有希望的节点。

特点：

- 使用限界函数评估节点
- 更快找到最优解

- 需要设计好的估价函数

6 分支限界的应用

6.1 0/1 背包问题

问题：从 n 个物品中选择，使得总重量不超过容量，总价值最大。

分支限界策略：

1. 按单位价值排序
2. 使用优先队列，优先级 = 当前价值 + 剩余最大可能价值
3. 剪枝：如果优先级 < 已知最优值，则不扩展

```
1  struct Node {
2      int level;        // 当前层
3      int profit;       // 当前价值
4      int weight;       // 当前重量
5      double bound;     // 上界
6  };
7
8  double calculateBound(Node u, int n, int W, vector<pair<int,int>>& items) {
9      if (u.weight >= W) return 0;
10
11     double profitBound = u.profit;
12     int j = u.level + 1;
13     int totWeight = u.weight;
14
15     while (j < n && totWeight + items[j].first <= W) {
16         totWeight += items[j].first;
17         profitBound += items[j].second;
18         j++;
19     }
20
21     if (j < n) {
22         profitBound += (W - totWeight) *
23             ((double)items[j].second / items[j].first);
24     }
25
26     return profitBound;
27 }
28
29 int knapsackBranchBound(int W, vector<pair<int,int>>& items) {
30     int n = items.size();
31     sort(items.begin(), items.end(),
32         [](const pair<int,int>& a, const pair<int,int>& b) {
33             return (double)a.second/a.first > (double)b.second/b.first;
34         });
35
36     priority_queue<Node> pq;
37     Node u, v;
```

```
38     u.level = -1; u.profit = 0; u.weight = 0;
39     u.bound = calculateBound(u, n, W, items);
40     pq.push(u);
41
42     int maxProfit = 0;
43
44     while (!pq.empty()) {
45         u = pq.top(); pq.pop();
46
47         if (u.bound <= maxProfit) continue;
48
49         // 扩展左子节点 (选择当前物品)
50         v.level = u.level + 1;
51         v.weight = u.weight + items[v.level].first;
52         v.profit = u.profit + items[v.level].second;
53
54         if (v.weight <= W && v.profit > maxProfit) {
55             maxProfit = v.profit;
56         }
57
58         v.bound = calculateBound(v, n, W, items);
59         if (v.bound > maxProfit) {
60             pq.push(v);
61         }
62
63         // 扩展右子节点 (不选择当前物品)
64         v.level = u.level + 1;
65         v.weight = u.weight;
66         v.profit = u.profit;
67         v.bound = calculateBound(v, n, W, items);
68         if (v.bound > maxProfit) {
69             pq.push(v);
70         }
71     }
72
73     return maxProfit;
74 }
```

6.2 旅行商问题 (TSP)

问题：找到访问所有城市并返回起点的最短路径。

分支限界策略：

1. 使用邻接矩阵表示图
2. 限界函数 = 当前路径长度 + 最小生成树下界
3. 优先队列按路径长度排序

NOTE

TSP 是 NP-hard 问题，分支限界可以在合理时间内找到中小规模问题的最优解。

7 回溯法与分支限界的比较

7.1 选择策略

使用回溯法：

- 需要找到所有解
- 解空间较小
- 约束条件复杂

使用分支限界：

- 只需要最优解
- 可以设计有效的限界函数
- 解空间较大

7.2 性能对比

方面	回溯法	分支限界
时间复杂度	指数级	指数级（但通常更快）
空间复杂度	$O(n)$	$O(2^n)$ （最坏）
实现难度	较简单	较复杂
适用场景	约束满足	优化问题

本章介绍了回溯法和分支限界两种系统搜索方法。回溯法适合寻找所有解，分支限界适合寻找最优解。选择合适的算法取决于具体问题的特点。

Chapter 19

搜索优化与高级技巧

Chapter 20

其他算法技巧



高级数据结构

21. 线段树与树状数组 ·····	97
22. 平衡树进阶 ·····	98
23. 分块与莫队算法 ·····	99
24. 并查集与高级应用 ·····	100
25. 字符串高级数据结构 ·····	101
26. 图论高级算法 ·····	102
27. 网络流与匹配 ·····	103
28. 其他高级数据结构 ·····	104

Chapter 21

线段树与树状数组

Chapter 22

平衡树进阶

Chapter 23

分块与莫队算法

Chapter 24

并查集与高级应用

Chapter 25

字符串高级数据结构

Chapter 26

图论高级算法

Chapter 27

网络流与匹配

Chapter 28

其他高级数据结构
